Contents

1 Digital Design

Reference basis: "Digital Logic Design by Morris Mano" - written/compiled by Maitreya Ranade.

- Digital Systems and Binary Numbers
- Boolean Algebra and Logic Gates
- Gate-Level Minimization
- Combinational Circuits
- Synchronous Sequential Logic
- Registers and Counters
- Memory and Programmable Logic
- Design at the Register Transfer Level
- Digital Integrated Circuits

---

2 Digital Systems and Binary Numbers

---

3 Introduction

- The binary information in a digital computer must have a physical existence in some medium for storing individual bits.
- A binary cell is a device that possesses two stable states and is capable of storing one bit (0 or 1) of information.
- Bit is a binary digit with only 2 discrete values 0 & 1.
- Bit is the smallest unit of data.
  - 1 nibble = 4 bits
  - 1 byte = 8 bits
  - 1 word = 16 bits = 2 bytes
  - 1 double word = 32 bits = 4 bytes

- Digital system is an interconnection of digital modules. These manipulate discrete quantities of information that are represented in binary form.

---

4 Number representation

Any number can be expressed in a specific number system. A general representation in a base-r system is:

$$a_n * r^n + a_{n-1} * r^{n-1} + ... + a_1 * r + a_0$$
$$+a_{-1} * r^{-1} + a_{-2} * r^{-2} + ...$$

where, - r = base of the number system, - a_i = coefficients of bases with values varying from 0 to r-1

4.1 Binary Addition

- The sum of 2 binary numbers is calculated by the same rules as in decimal.
- Any carry obtained in a given significant position is used by the pair of digits one significant position higher.
- Binary addition for a single bit is:
  - 0+0 = sum: 0 & carry:0
  - 1+0 or 0+1 = sum: 1 & carry:0
  - 1+1 = sum: 0 & carry:1

4.2 Binary Subtraction

The subtraction of 2 binary numbers is calculated by the same rules as in decimal except that the borrow in a given significant position adds 2 to a minuend digit.

4.3 Binary Multiplication

In binary multiplication, the result is onn only when both the input bits are 1. It looks like this: - 0x0 = 0 - 1x0 or 0x1 = 0 - 1x1 = 1

4.4 Binary Division

The division of 2 binary numbers is calculated by the same rules as in decimal.

---

5 Number base conversion

If a number includes a radix point, the number is split into an integer and a fraction part. Then conversion of decimal integer to a number in base-r is done by dividing the number and all successive quotients by r and accumulating the reminders. In case of fractions, multiplication is used instead of division.

5.1 Case 1: Higher Base to Lower Base

(e.g., Decimal to Binary, Hex to Binary)

5.1.1 Process

- Convert the given number from Base 1 to **decimal (base 10)** first (if needed).
- Then convert decimal to Base 2 using:
  - **Integral part**: repeated division by Base 2
  - **Fractional part**: repeated multiplication by Base 2

5.1.2 Integral Part Conversion

- Divide the number repeatedly by Base 2.
- Record remainders.
- Final result = **remainders read in reverse order**.

Example (Decimal to Binary):

$$25_{10} = 11001_2$$

- 25 ÷ 2 -> remainder 1
- 12 ÷ 2 -> remainder 0
- 6 ÷ 2 -> remainder 0
- 3 ÷ 2 -> remainder 1
- 1 ÷ 2 -> remainder 1

5.1.3 Fractional Part Conversion

- Multiply fractional part repeatedly by Base 2.
- Record integer part at each step.
- Final result = **digits read in order**.

Example:

$$0.625_{10} = 0.101_2$$

$0.625 \times 2 = 1.25$ -> carry 1
$0.25 \times 2 = 0.5$ -> carry 0
$0.5 \times 2 = 1.0$ -> carry 1

5.1.4 Final Combined Result

$$25.625_{10} = 11001.101_2$$

5.2 Case 2: Lower Base to Higher Base

(e.g., Binary to Decimal, Binary to HEX)

5.2.1 Process

- Expand number using positional weights of Base 2.
- Separate **integral and fractional parts**.
- Use powers of 2 for conversion.

5.2.2 Integral Part Conversion

- Multiply each digit by corresponding power of 2.
- Sum all terms.

Example:

$$11001_2 = (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= 16 + 8 + 0 + 0 + 1 = 25_{10}$$

5.2.3 Fractional Part Conversion

- Multiply each digit by negative powers of 2.

Example:

$$0.101_2 = (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})$$

$$= 0.5 + 0 + 0.125 = 0.625_{10}$$

5.2.4 Final Combined Result

$$11001.101_2 = 25.625_{10}$$

5.3 Alternative methods for number base conversion

5.3.1 Decimal Binary Conversion

- Decimal to Binary Conversion: There are 2 methods for this:
  - By division:
    - Divide by 2
    - Mark the remainder
    - Divide the quotient by 2

    - * Append to the remainder
    - * Repeat this until quotient < 2
    - * For fractional part, multiply by 2
  - By subtraction:
    - * Take highest power of 2 which is lesser than or equal to the decimal number
    - * Subtract it from the number
    - * Repeat this until the remainder becomes zero
    - * Collect all the powers of 2 subtracted from the number to compute the bits
    - * For fractional part, add by powers of 2
- Binary to Decimal Conversion: Add weighted powers of 2 just like the number representation

5.3.2 Decimal Octal & Hexadecimal Conversion

- Exactly similar to binary just, use the base as 8, 16 respectively.

5.3.3 Binary Octal Conversion

- Binary to Octal Conversion: Clubbing 3 bits to obtain a single Octal digit.
- Octal to Binary Conversion: Replace octal digit into corresponding 3 binary digits.

5.3.4 Binary Hexadecimal Conversion

- Binary to HEX Conversion: Clubbing 4 bits to obtain a single Hex digit.
- HEX to Binary Conversion: Replace Hex digit into corresponding 4 binary digits.

---

6 Complements of numbers

Complements are used in digital systems to simplify the subtraction operation and logical manipulation. There are 2 types of complements:

1. **Diminished radix complement ((r-1)'s complement):** (r-1)'s complement of an n-digit number N in base-r is defined as: $> (r^n - 1) - N$ where, r is base number.
   - 9's complement = subtracting every digit from 9
   - 1's complement = subtracting every digit from 1
     - which is same as toggling bits
2. **Radix complement (r's complement):** r's complement of an n digit number 'N' in base-r is defined as: $> (r^n - N)$ for N != 0

   '0' for N = 0

   r's complement = (r-1)'s complement + 1 Complement of the complement restores the number to its original value.

- For eg. 9's complement of 546 is 453
- For eg. 10's complement of 546 is 454 = 453+1
- For eg. 1's complement of 1100 is 0011
- For eg. 2's complement of 1100 is 0100 = 0011+1

6.1 Subtraction using complements

Consider M & N are 2 n-digit unsigned numbers in base-r. > M - N = M + (r's complement of N)

if M $\geqslant$ N : > M - N = M + (r's complement of N) + r^n; Carry can be ignored.

if M < N : > M - N = r^n - (N- M) > > = r's complement of (N-M) (answer) result is negative. > > = - (r's complement of answer) (familiar representation) > > Final carry bit acts as a sign bit for the answer. > - If carry bit = 1, then the result is positive. > - If carry bit = 0, then the result is negative.

- For eg.
  - Decimal number:
    - 530 - 240
      - = 530 + (10's complement of 240)
      - = 530 + (759+1)
      - = (1)290 = 290 (neglecting carry)
    - 240 - 530
      - = - (10's complement of (240 + (10's complement of 530)))
      - = - (10's complement of (240 + (469+1)))
      - = - (10's complement of 710)
      - = - (289+1) = -290
  - Binary number:
    - 1100 - 1010
      - = 1100 + (2's complement of 1010)
      - = 1100 + (0101+1)
      - = 1100 + 0110
      - = (1)0010 = 0010
    - 1010 - 1100
      - = - (2's complement of (1010 + (2's complement of 1100)))
      - = - (2's complement of (1010 + (0011+1)))
      - = - (2's complement of 1110)
      - = - (0001+1) = - (0010)

---

7 Signed Binary Numbers

7.1 Ordinary Arithmatic

In ordinary arithmatic, negative numbers are represented with a minus sign. But due to hardware limitations, Computers must represent everything with binary digits. Signed Binary Numbers representation: - sign bit = 0 for a positive number - sign bit = 1 for a negative number

sign bit | Number (magnitude) |

- Unsigned number: Left most bit is the MSB
- Signed number: Left most bit is the sign bit.
- Eg. 01001 = 9 (unsigned) & + 9 (signed)
- Eg. 11001 = 25 (unsigned) & -9 (signed)

This representation is known as the signed magnitude convention, or ordinary arithmatic.

7.2 Signed Complement System

In this system, a negative number is indicated by it's complement. Representation of a number in Signed Complement System :

sign bit | Complement |

There are 2 representations of the Signed Complement System: - Signed 1's complement - Signed 2's complement

Complement could be (1's or 2's) but usually 2's complement. - For eg. '-9' has three different representations: - Signed magnitude: 1-0001001 (Changing the leftmost bit) - Signed 1's compliment: 1-1110110 (Complement of all the bits including sign bit) - **Preferred method** : Signed 2's compliment: 1-1110111 (Complement of all the bits & add 1) - Range of the number representations: - Signed magnitude: $-2^{n-1}+1$ to $2^{n-1}-1$ for ex. -7 to 7 for 4 bit numbers - Signed 1's compliment: $-2^{n-1}+1$ to $2^{n-1}-1$ for ex. -7 to 7 for 4 bit numbers - Signed 2's compliment: -2^{n-1} to $2^{n-1}-1$ for ex. -8 to 7 for 4 bit numbers

An extra number -2^{n-1} is accommodated in 2's compliment representation.

In signed magnitude's compliment, zero is represented as: - Positive zero: 0 for ex. 0000 - Negative zero: -0 for ex. 1000

In signed 1's compliment, zero is represented as: - Positive zero: 0 for ex. 0000 - Negative zero: -0 for ex. 1111

In signed 2's compliment, zero is represented as: - Positive zero: 0 for ex. 0000 - Negative zero: Doesn't exist - Extra negative number: -2^{n-1} for ex. -8 looks like 1000

Positive numbers are same in all the representations.

7.3 Addition of signed numbers

- Signed Magnitude:
  - if signs are same:
    - Subtract magnitude.
    - Keep the sign.
  - if signs are different:
    - Compare the numbers.
    - Subtract smaller one from the larger one.
    - Append sign of the larger number.
- Signed 2's complement: simply add the numbers including sign bit and discard the carry.

7.4 Subtraction of signed numbers

- 2's complement of subtrahend (including sign bit) is added to the minuend (including sign bit) and discard the carry.
- As subtraction & addition basically use the same method in a signed 2's complement representation.
- Computers need only one common hardware circuit to handle both types of arithmatic.

---

8 Binary codes

Code is nothing but group of symbols.

8.1 Binary coded Decimal code (BCD)

Decimal number representation in binary (4 bits are used to represent a decimal digit). Binary combination 0000 to 1001 represent 0-9 whereas, 1010 to 1111 are not used and have no meaning in BCD.

8.1.1 BCD addition

1. Sum $\leqslant$ 1001 (without carry)
   Sum = A + B
2. Sum $\geqslant$ 1010
   Sum = add 6 (16-10) to the binary sum and also generates a carry
   Representation of signed BCD is similar to the signed numbers in binary.

8.2 Gray code

Advantage over straight binary numbers only one bit change in the next an increment. This is useful in continuous data such as ADCs

Binary Number	Gray Code
0	00
1	01
2	11
3	10

8.3 ASCII character code

- ASCII stands for American Standard code for Information Interchange.
- An alphanumeric character set representation with binary digits.
- This set includes 10 decimal digits, 26 letters of the alphabet (uppercase & lowercase) and multiple special characters.
- This is a 7 bit code but used as a byte for storing in computers.

8.4 Error detecting code

Eighth bit of the ASCII code is used to detect errors. The bit is called as parity bit. Parity bit represents total of 1's in the number. The parity could be odd or even.

Number	Even Parity	Odd Parity
101	0101	1011
111	1111	0111

9 Binary storage and registers

- The binary information in a digital computer must have a physical existence in some medium for storing bits.
- A binary cell is a device that possesses 2 stable states and is capable of storing one bit of information (0 or 1).

9.1 Registers

A register is a contiguous group of binary cells. An n-bit register can store n bit information, and can have 2^n possible states.

9.2 Register transfer

A digital system is characterized by its registers and the components that perform data processing. >**The device most commonly used for holding data is a register, and Binary variables are manipulated by means of digital logic circuits.** - In digital systems, a register transfer operation is a basic operation that consists of a transfer of binary information from one set of registers into another set of registers. - The transfer may be direct, from one register to another, or may pass through data-processing circuits to perform an operation.

---

10 Binary logic

- Binary logic deals with variables that take on 2 discrete values and with operations that assume logical meaning.
- Binary logic consists of binary variables and a set of logical operation.
- Each variables has only 2 distinct values 1 & 0.
- Basic logical operations include only 3 functions: AND, OR & NOT.

10.1 Logic gates

- Logic gates are the electronic circuits that operate on one or more physical input signals to produce an output signal.
- Binary information is stored into voltage levels.
- Voltage ranges are assigned to logic 0 & Logic 1 for interpretation.
- Basic gates: NOT, AND, & OR (with these gates we can create all the other gates)
- Universal gates: NAND, & NOR
- Arithmatic gates: XOR, & XNOR

---

11 Boolean algebra and Logic gates

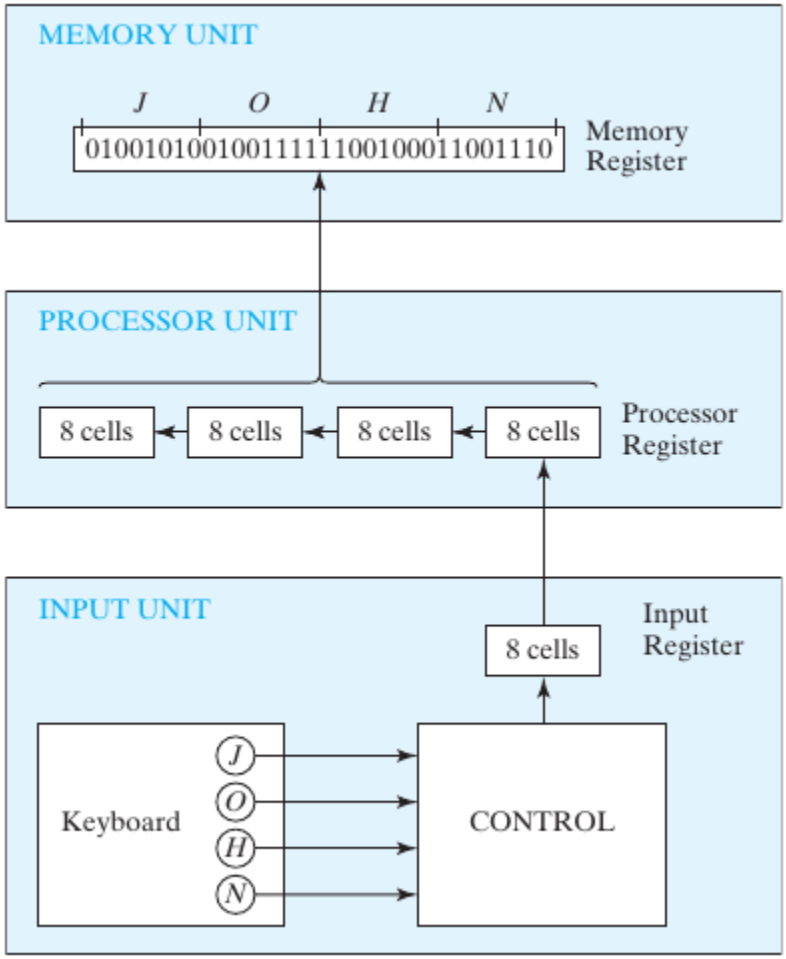
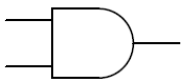
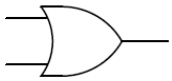


Figure 1: Transfer of information among registers



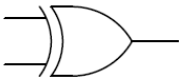
AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



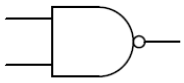
OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



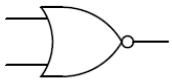
XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



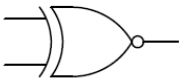
NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Figure 2: Logic Gates & their Truth tables

12 Introduction

- Boolean algebra was introduced by George Boole.
- Boolean Algebra comprises of set of rules used to simplify the given logic expression without changing it's functionality.
- The most Common postulates used to formulate various algebraic structures are:

1. Closure: A set S is closed with respect to a binary operator if, for every pair of elements of S, the binary operator specifies a rule for obtaining a unique & element of S.
2. Associative law:$ (x*y*)z = x(*yz*) $ for all $ x,y,z S $
3. Commutative law:$ x*y* = *y*x $ for all $ x,y S $
4. Identity element:$ e*x* = *x*e = x $ for every $ x S $
5. Inverse: A set S having the identity element e - is said to have an inverse when ,$ x*y=e $
6. Distributive law:$ x*(y.z)* = *(x*y).(x*z) $

13 Basic theorems & Properties of boolean algebra

13.1 Duality

If the operators and the elements are interchanged, every algebraic expression deducible from the postulates of boolean algebra remains unchanged.

13.2 Basic theorems

- Postulate 2:
  - $ x + 0 = x $
  - $ x . 1 = x $
- Postulate 5:
  - $ x + x' = 1 $
  - $ x . x' = 0 $
- Theorem 1
  - $ x + x = x $
  - $ x . x = x $
- Theorem 2
  - $ x + 1 = 1 $
  - $ x # 0 = 0 $
- Theorem 3, involution
  - $ (x ')' = x $
- Postulate 3, commutative
  - $ x + y = y + x $
  - $ xy = yx $
- Theorem 4, associative
  - $ x + (y + z) = (x + y) + z $
  - $ x(yz) = (xy)z $
- Postulate 4, distributive
  - $ x(y + z) = xy + xz $
  - $ x + yz = (x + y)(x + z) $
- Theorem 5, DeMorgan
  - $ (x + y)' = x 'y' $
  - $ (xy)' = x ' + y' $
- Theorem 6, absorption
  - $ x + xy = x $
  - $ x(x + y) = x $

13.3 Summary operations:

1. Compliment: $ (A')' = A $
2. AND:
   1. $ A.A = A $
   2. $ A.0 = 0 $
   3. $ A.1 = A $
   4. $ A.A' = 0 $
3. OR:
   1. $ A+A = A $

2. $ A+0 = A $
3. $ A+1 = 1 $
4. $ A+A' = 1 $

4. Distributive:
   1. $ A+BC = (A+B).(A+C) $
   2. $ A.(B+C) = A.B + A.C $
   3. $ A+A'B = A+B $
   4. $ A'+AB = A'+B $
5. De Morgan's law:
   1. $(A+B)' = A'.B' $
   2. $(A.B)' = A'+B' $

13.4 Operator Precedence

1. Parentheses
2. NOT
3. AND
4. OR

14 Boolean functions

A boolean function is described by an algebraic expression consisting of binary variables, the constants (0 & 1) and the logic operation symbols. For eg. $ F_1 = x + y'z $

A boolean function expresses the logical relationship between binary variables & is evaluated by determining the binary value of the expression for all possible values of the variables.

14.1 Truth table

- A boolean function can be represented in a truth table.
- Truth table enlists all the possible combinations 2^n of the variables (n), and the corresponding output value of boolean function.
- A boolean function can be transformed into a circuit diagram composing logic gates from algebraic expression which is also known as schematic.

---

15 Canonical & standard forms

15.1 Minterms and Maxterms

15.1.1 Minterm or standard product

- The product of all variables (.), either with or without complement, is known as minterm.
- All variables forming an AND term, with or without complement, provide 2n possible combinations, called minterms

15.1.2 Maxterms or standard sums

- The sum of all variables (+), either with or without complement, is known as maxterm.

- All variables forming an OR term, with or without complement, provide 2n possible combinations, called maxterms
- Shorthand notations:
  - minterm: m_j
  - maxterm: M_j
  - where, j is the decimal equivalent of the binary number of the minterm designated.
- Sample Minterms and Maxterms for Three Binary Variables look like the following:

![Minterms and Maxterms for Three Binary Variables](images/MinMaxterms.png " Minterms and Maxterms for Three Binary Variables")

> **A Boolean function can be expressed by forming a minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.**

- Say, we have a boolean expression = $ f_{1} $
- Firstly, obtain the truth table of the function directly from the algebraic expression
- Read the minterms from the truth table that produce a 1 in the $ f_{1}$
- Take OR of all those minterms.

For example: $ f_{1} = x'y'z + xy'z' + xyz = m_{1}+m_{4}+m_{7} = (1,4,7) $

> **Similarly, a Boolean function can be also expressed by forming a maxterm for each combination of the variables that produces a 0 in the function and then taking the AND of all those terms.**

- Similar to minterms, we have a boolean expression = $ f_{1} $
- Firstly, obtain the truth table of the function directly from the algebraic expression.
- Read the maxterms from the truth table that produce a 0 in the $ f_{1} $
- Take AND of all those minterms.

For example: $ f_{1} = (x+y+z)(x+y'+z)(x'+y+z')(x'+y'+z) = M_{0}.M_{2}.M_{3}.M_{5}.M_{6} = (0,2,3,5,6) $

> **Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form.**

15.2 Conversion between Canonical Forms

The complement of a function expressed as the sum of minterms equals the sum of minterms missing from the original function. $ m'*{j} = M*{j} $

15.3 Standard Forms

- In canonical form, minterm or maxterm must contain, all the variables, either with or without complement.
- Another way to express Boolean functions is in **standard form**.
- In this form, the terms that form the function may contain one, two, or any number of literals/variables.
- For example, $ F_{1} = y' + xy + x'yz' $
- This standard type of expression results in a two-level structure of gates. (Either group of OR gates followed by an AND gate or group of AND gates followed by an OR gate)

15.4 Nonstandard Forms

- A Boolean function may be expressed in a nonstandard form.
- The function is neither in sum-of-products nor in product-of-sums form.
- For example, $ F_{1} = AB + C(D + E) $

---

16 Digital logic gates

- A gate can be extended to have multiple inputs if the binary operation it represents is commutative and associative.

---

17 Integrated Circuits

An integrated circuit (IC) is fabricated on a die of a silicon semiconductor crystal, called a chip, containing the electronic components for constructing digital gates.

17.1 Levels of Integration

Digital ICs are categorized according to the complexity of their circuits as measured by the number of logic gates in a single package.

- SSI (Small Scale Integration): Number of gates are less than 10.
- MSI (Medium Scale integration): Number of gates are between 10 to 1000 Gates.
- LSI (Large Scale Integration): Number of gates are in thousands eg. Processors, memory chips & programmable logic devices.
- VLSI (Very Large Scale Integrations): Number of gates are in millions ex. complex microcomputer chips.

17.2 Digital Logic Families

- Digital integrated circuits are also classified by their specific circuit technology i.e their digital logic family.
- The electronic components employed in the construction of the basic circuit are usually used to name the technology.
  - TTL transistor–transistor logic (standard)
  - ECL emitter-coupled logic (systems requiring high-speed operation)
  - MOS metal-oxide semiconductor (suitable for circuits that need high component density)
  - CMOS complementary metal-oxide semiconductor (systems requiring low power consumption, hence dominant in the VLSI industry)

---

18 Computer Aided Design of VLSI circuits

- Integrated circuits having sub-micron geometric features are manufactured by optically projecting light onto silicon wafers.

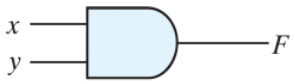
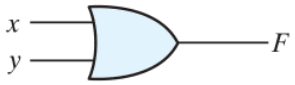
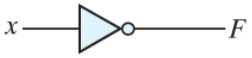
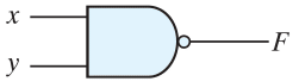
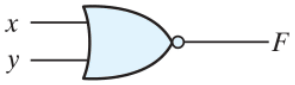
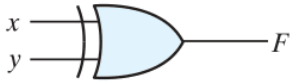
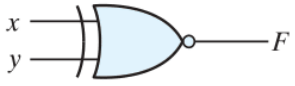
Name	Graphic symbol	Algebraic function	Truth table
AND		$F = x \cdot y$	x y | F 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$F = x + y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter		$F = x'$	x | F 0 | 1 1 | 0
Buffer		$F = x$	x | F 0 | 0 1 | 1
NAND		$F = (xy)'$	x y | F 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR		$F = (x + y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

FIGURE 2.5
Digital logic gates

Figure 3: some discription

- The design of digital systems with VLSI circuits containing millions of gates is an enormous task. Only with the help of CAD tools, this task is made possible considering system complexity.
- EDA (Electronic Design Automation) automates multiple steps of design phases of ICs.
- Typical design flow for creating VLSI circuits:
  1. Design entry: Schematic / HDL based model
  2. Phyical Realisation
     1. ASIC
     2. FPGA
     3. PLD
     4. Full Custom IC
  3. Hardware fabrication with CAD & EDA (Schematic entry)
- An important development in the design of digital systems is the use of a hardware description language (HDL).
- HDL description of a circuit's functionality can be abstract, without reference to specific hardware, thereby freeing a designer to devote attention to higher level functional detail
- HDL-based models of a circuit or system are simulated to check and verify its functionality before it is submitted to fabrication, thereby reducing the risk and waste of manufacturing a circuit that fails to operate correctly.
- Two HDLs—Verilog and VHDL—have been approved as standards by the Institute of Electronics and Electrical Engineers (IEEE) and are in use by design teams worldwide.

19 Gate-Level Minimization

---

20 Introduction

Gate-level minimization is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit.

- It is difficult to perform by manual methods for multi input logic.
- Computer-based logic synthesis tools can minimize a large set of Boolean equations efficiently and quickly.
- It is important that a designer understand the underlying mathematical description and solution of the problem.

---

21 The K-Map Method

- Boolean expressions may be simplified by algebraic means.
- However, the map method provides a simple, straightforward & more efficient procedure for minimizing Boolean functions.
- This method may be regarded as a pictorial form of a truth table. The map method is also known as the **Karnaugh map** or **K-map**.
- The map representation uses this basic property: "Any two adjacent(**only horizontal or vertical and not diagonal**) squares in the map differ by only one variable."

21.1 Two-Variable K-Map

There are four minterms for two variables. Hence, the map consists of four squares, one for each minterm.

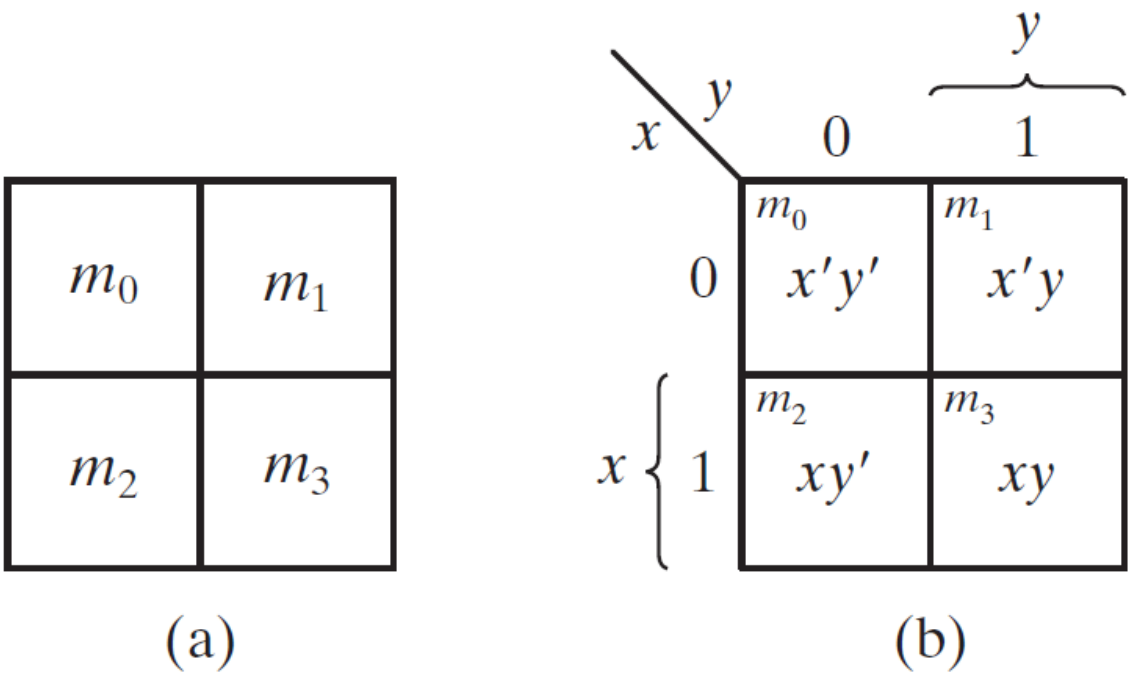


Figure 4: Two-variable K-map

21.2 Three-Variable K-Map

- There are eight minterms for three binary variables; therefore, the map consists of eight squares.
- Note that the minterms are arranged, not in a binary sequence, but in a sequence similar to the Gray code.
- The characteristic of this sequence is that only one bit changes in value from one adjacent column to the next.

21.3 Four-Variable K-Map

- The K-Map for Boolean functions of four binary variables (w, x, y, z) as shown in the figure below have 16 minterms.
- The rows and columns are numbered in a Gray code sequence, with only one digit changing value between two adjacent rows or columns.

21.4 Five-Variable K-Map

- Maps for more than four variables are more complex. A five-variable map needs 32 squares and a six-variable map needs 64 squares.

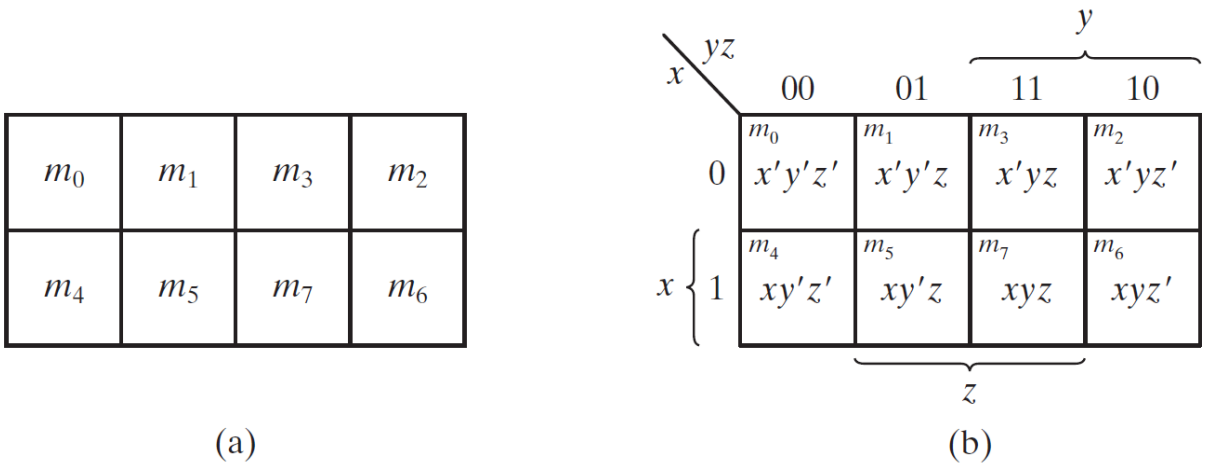


Figure 5: Three-Variable K-Map

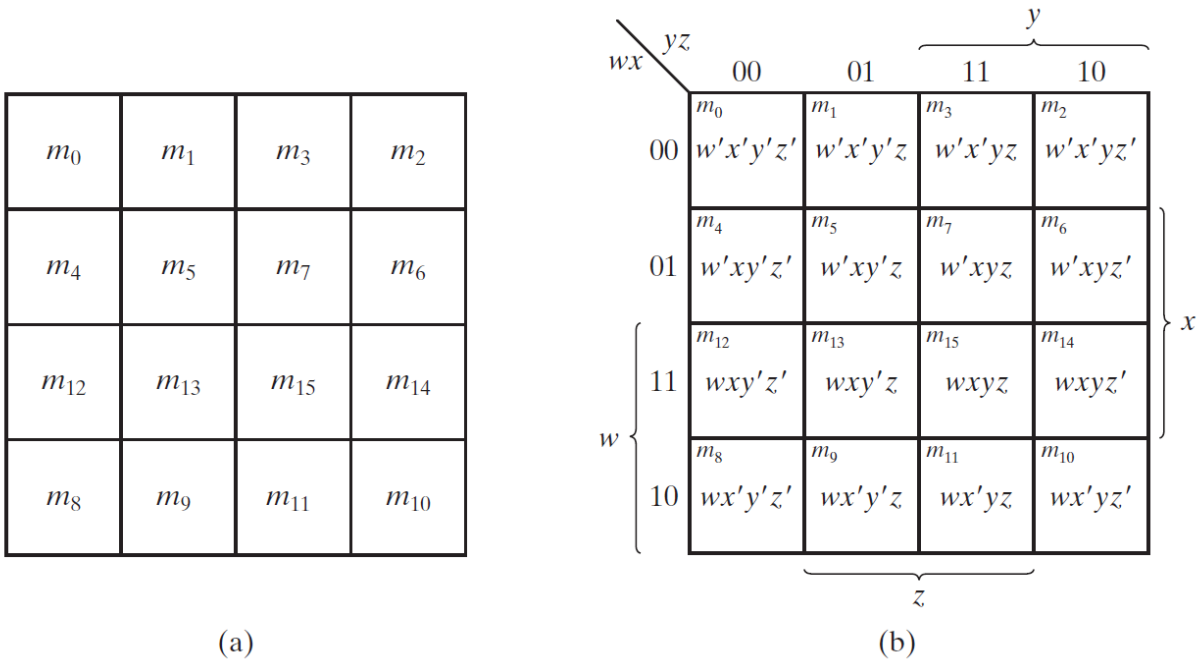


Figure 6: Four-Variable K-Map

- When the number of variables becomes large, the number of squares becomes excessive and the geometry for combining adjacent squares becomes more involved.
-

21.5 Minimizing with K-Map

The procedure for minimizing a Boolean function with a K-map is explained below:

1. First, a "1" is marked in each minterm square that represents the function.
2. The next step is to find possible adjacent squares.
3. Add the minterms within the square.
    1. Any two minterms in adjacent squares (vertically or horizontally, but not diagonally) that are ORed together will cause a removal of the dissimilar variable.
4. Add the product terms from all possible squares.
5. The logical sum of the product terms gives the simplified expression.

Following example will explain the procedure for minimizing a Three-Variable Boolean function with a Three-Variable K-map.

- Sample question: For the Boolean function F = A'C + A'B + AB'C + BC

(a) Express this function as a sum of minterms.
(b) Find the minimal sum-of-products expression.

- Answer:
  - The function can be expressed in sum-of-minterms form as F = A'C + A'B + AB'C + BC = $\Sigma(1, 2, 3, 5, 7)$
  - Final solution derived from the K-Map is: F = A'C + A'B + AB'C + BC = C + A'B

21.6 Important points to note for K-Map method

- One square represents one minterm, giving a term with four literals.
- Two adjacent squares represent a term with three literals.
- Four adjacent squares represent a term with two literals.
- Eight adjacent squares represent a term with one literal.
- Sixteen adjacent squares produce a function that is always equal to 1.
- The result is minimized but may not be unique.

21.6.1 Prime Implicants

- In choosing adjacent squares in a map, we must ensure that:
  - All the minterms of the function are covered before we combine the squares.
  - The number of terms in the expression is minimized
  - There are no redundant terms (i.e., minterms already covered by other terms).
- The group of combined minterms is referred to as an Implicant.
- A prime implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map.
- The prime implicants of a function can be obtained from the map by combining all possible maximum numbers of squares.
- If a minterm in a square is covered by only one prime implicant, that prime implicant is said to be essential prime implicant.

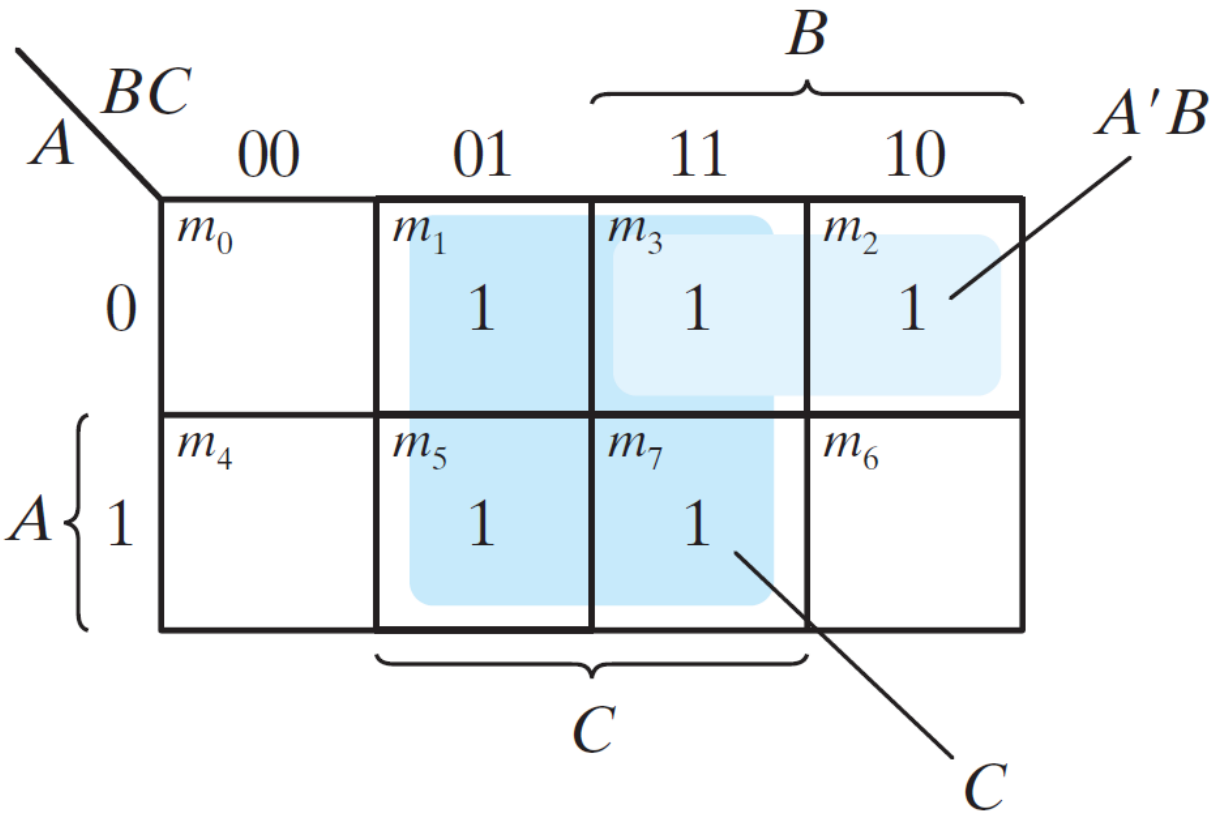


Figure 7: K-Map of example

- The following figure shows examples of Prime implicants:

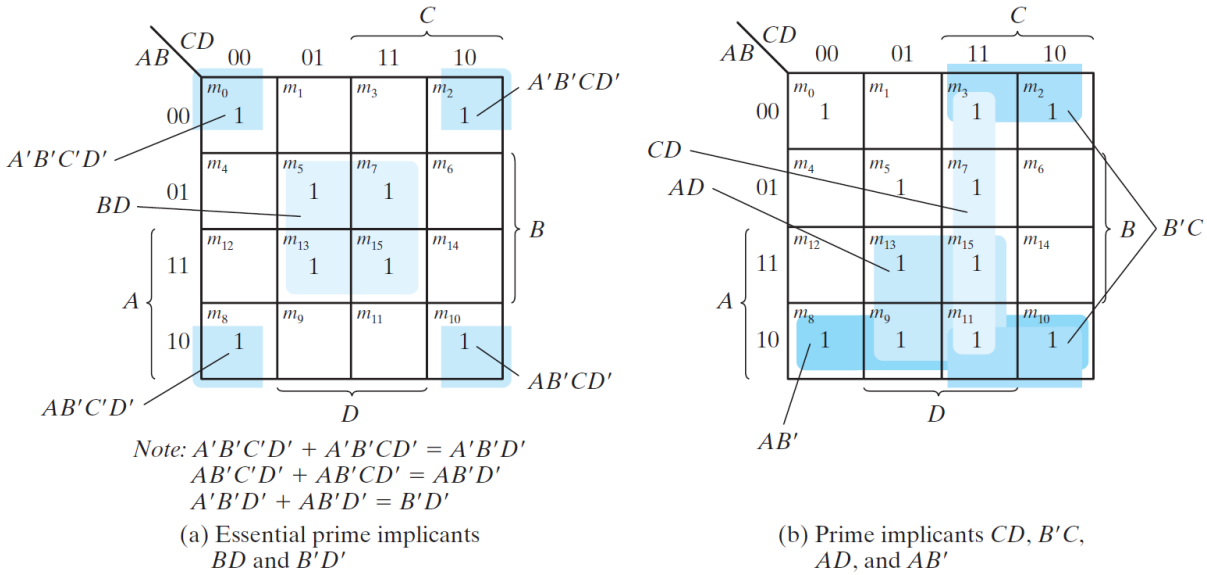


Figure 8: Example of Prime Implicants

---

22 Product-of-Sums Simplification

- The minimized Boolean functions derived from the map in all previous examples were expressed in sum-of-products form. With a minor modification, the product-of-sums form can be obtained.
- We mark the empty squares by 0's and combine them into valid adjacent squares.
- With this, we obtain a simplified sum-of-products expression of the complement of the function(F').
- The complement of F' gives us back the function F in product-of-sums form (a consequence of DeMorgan's theorem).
- Following example shows the 2 representations of the same function.
- Consider a function, F (A, B, C, D) = $\Sigma(0, 1, 2, 5, 8, 9, 10)$
- Answer:
  - Combining the squares with 1's gives the simplified function in sum-of-products form: F = B'D' + B'C' + A'C'D
  - Combining the squares with 0's and applying DeMorgan's theorem gives the simplified function in product-of-sums form: F=(A'+B')(C'+D')(B'+D)

Hence, a function can be implemented in 2 ways:

1. With a group of AND gates, one for each AND term. The outputs of the AND gates are connected to a single OR gate.

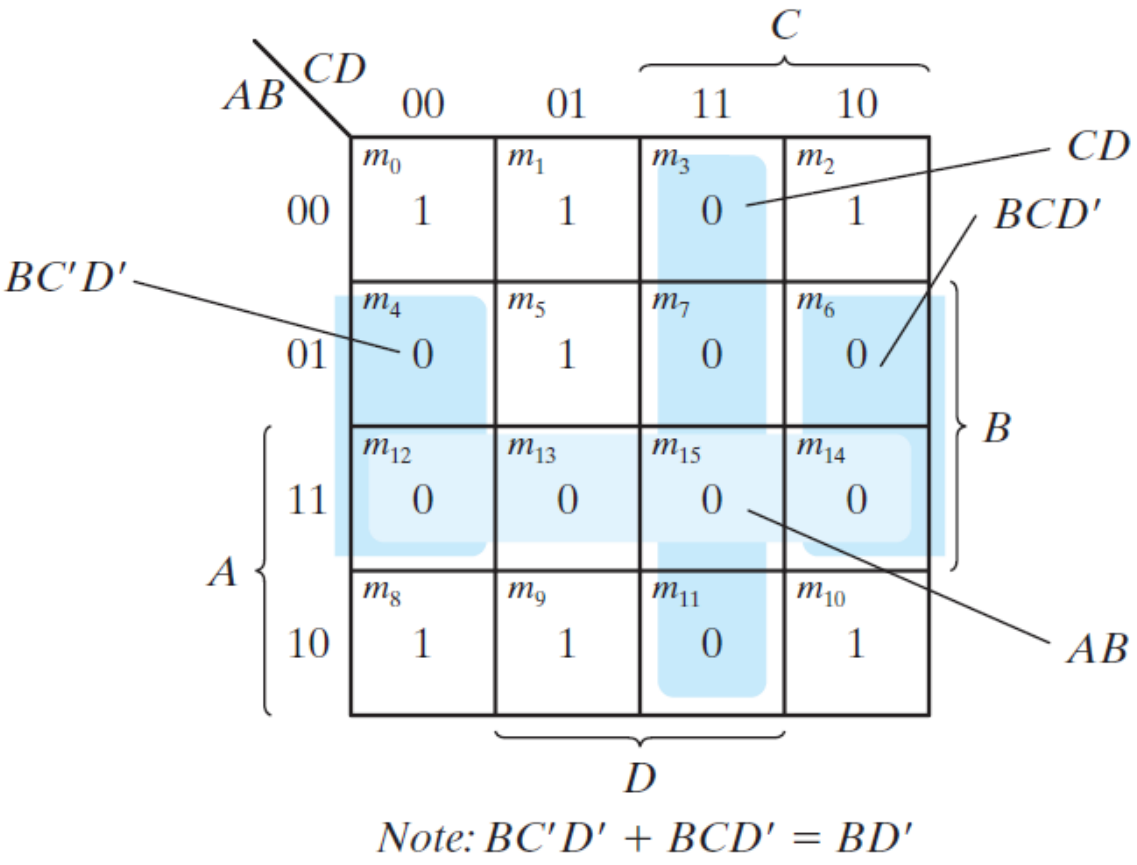


Figure 9: Example of Product-of-Sums form

2. With a group of OR gates, one for each OR term. The outputs of the OR gates are connected to a single AND gate.

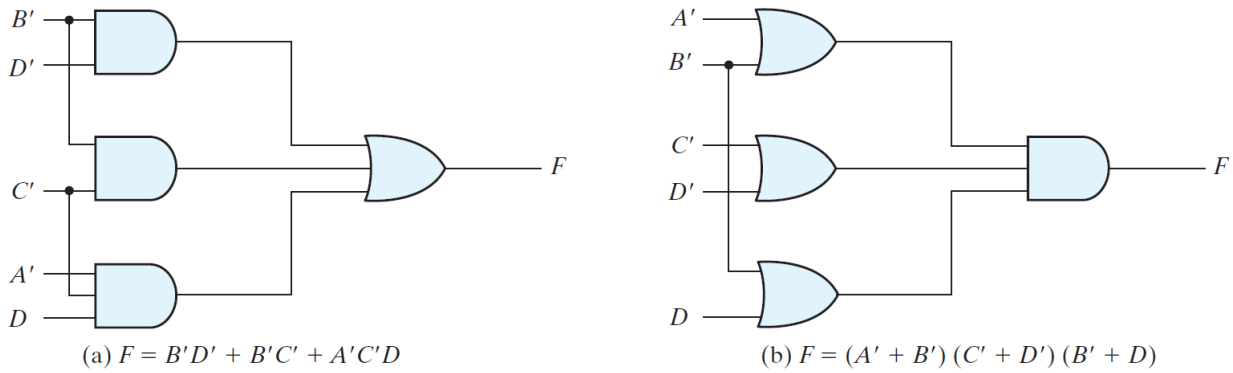


Figure 10: Gate implementations

- Either configuration (sum-of-products or product-of-sums) forms two levels of gates.
- Hence, the implementation of a function in a standard form is said to be a two-level implementation.
- The two-level implementation may not be practical, depending on the number of inputs to the gates.

---

23 Don't-Care Conditions

- In some applications, function is not specified for certain combinations of the variables.
- Functions that have unspecified outputs for some input combinations are called incompletely specified functions.
- The unspecified minterms of a incompletely specified function is known as **don't-care condition**.
- To distinguish the don't-care condition from 1's and 0's, an X is used.
- While evaluating the K-Map, the don't-care minterms may be assumed to be either 0 or 1.

---

24 NAND and NOR Implementation

- Digital circuits are frequently constructed with NAND or NOR gates rather than with AND and OR gates.
- NAND and NOR gates are easier to fabricate with electronic components and are the basic gates used in all IC digital logic families.
- Due to this, Boolean functions are converted from AND, OR, and NOT into equivalent NAND and NOR logic diagrams.

24.1 NAND Circuits

The NAND gate is said to be a **universal gate** because any logic circuit can be implemented with it.

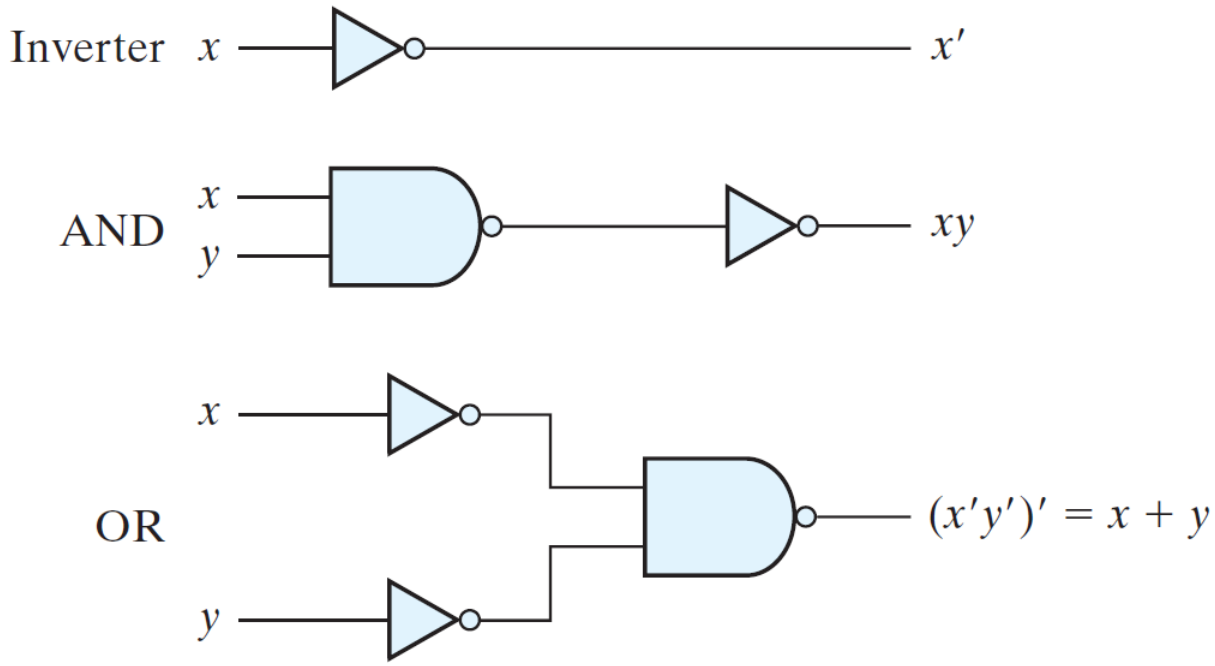


Figure 11: Logic operations with NAND gates

24.1.1 Two-Level Implementation

The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form. The implementation with AND and OR gates can easily be realized with the use of NAND gates only.

24.2 NOR Implementation

- The NOR operation is the dual of the NAND operation.
- The NOR gate is another universal gate that can be used to implement any Boolean function.
- The complement operation is obtained from a one input NOR gate that behaves exactly like an inverter.
- The OR operation requires two NOR gates, and the AND operation is obtained with a NOR gate that has inverters in each input.

24.2.1 Two-Level Implementation

A two-level implementation with NOR gates requires that the function be simplified into product-of-sums form. The implementation with AND and OR gates can easily be realized with the use of NOR gates only.

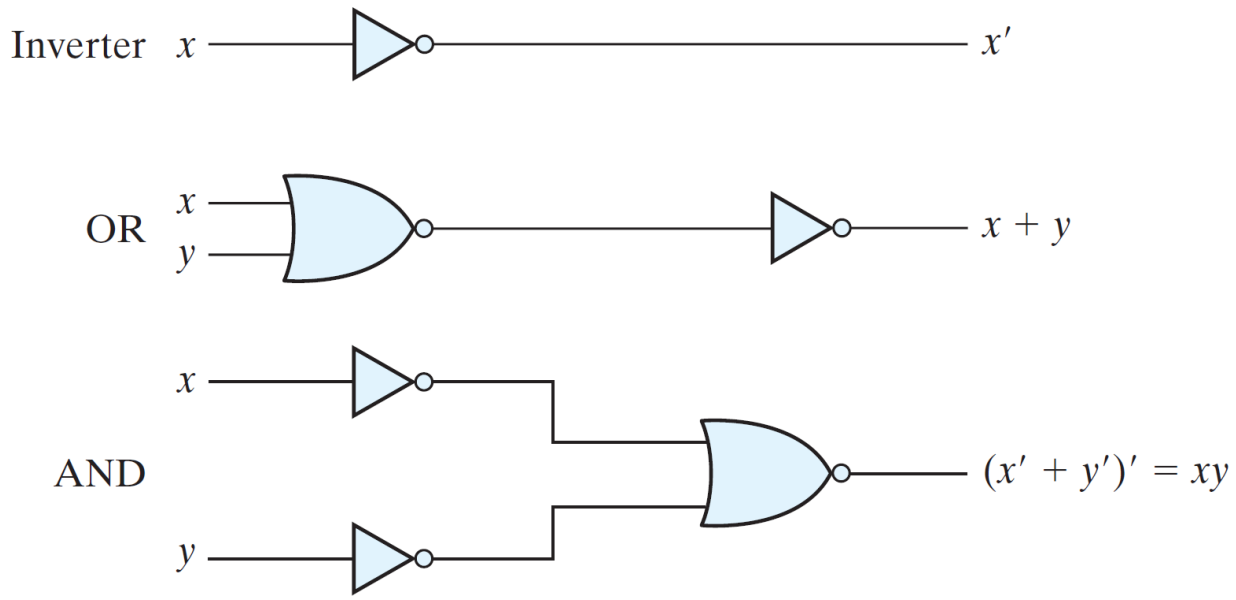


Figure 12: Logic operations with NOR gates

25 Exclusive-OR Function

The exclusive-OR (XOR), denoted by the symbol ⊕, is a logical operation that performs: x ⊕ y = xy' + x'y

- The exclusive-OR signifies when both the inputs are not equal.
- XOR is useful in arithmetic operations and error detection and correction circuits.

---

26 Combinational Logic

---

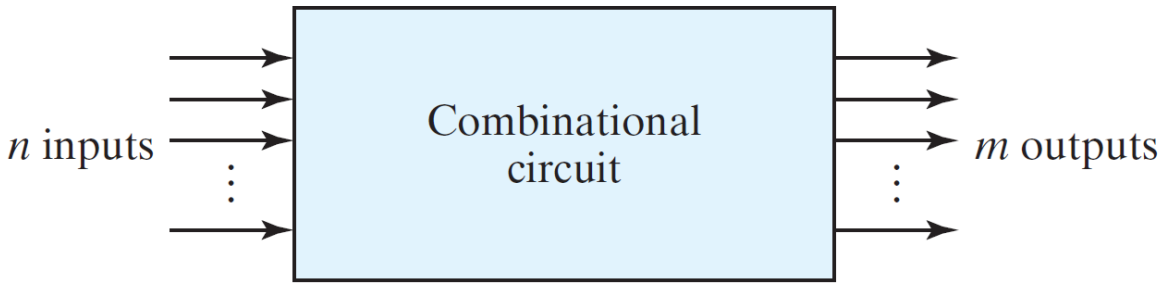
27 Introduction

- Logic circuits for digital systems maybe combinational or sequential.
- Combinational circuits employ boolean functions whereas, sequential circuits employ storage element in addition to logic gates.

27.1 Combinational circuits

- A logic circuit is combinational if it's outputs are at any time are a function of only the present inputs.
- A combinational circuit consists of an interconnection of logic gates.
- Combinational logic gates process the input signals to produce the value of the output signal.

- A combinational circuit can be specified with a truth table that lists the output.



27.2 Sequential circuits

- Outputs of the sequential circuits are a function of the inputs and the state of the storage elements.
- Hence, the outputs of a sequential circuit depend at any time on not only the present values of inputs but also on past inputs.

---

28 Analysis of combinational circuits

- The logic diagram of a combinational circuit has logic gates with no feedback paths or memory elements.
- Analysis of a combinational circuit determines its functionality i.e. the logic function that the circuit implements.

Analysis of combinational circuits can be done through the following 2 methods:

- Method 1:
  1. Make sure that the circuit is combinational & not Sequential (without memory element & feedback loops).
  2. Obtain the output boolean functions or the truth table.
  3. Use hierarchy and input, output to add intermediate primary logic gates to determine the boolean functions.
  4. Derive truth table: Assign a value to the output base of on every input combination.
- Method 2:
  1. Develop HDL model of the circuit.
  2. Write a test-bench (logic simulation)
  3. Step 1 still requires method 1

---

29 Design procedure

These are the steps involved in the design procedure of combinational circuits:

1. Gather specifications of the circuit.
2. Determine number of inputs & outputs.

3. Derive truth table.
4. Obtain simplified boolean functions.
5. Draw the logic diagram & verify correctness of the design. (Manually or by simulation.)

---

30 Binary Adder

- A binary adder performs an addition of 2 bits.
- Extra bit needed for the addition which is the higher significant bit of the addition is called a "Carry".
- The result of addition apart from the carry is called "Sum".

30.1 Half Adder

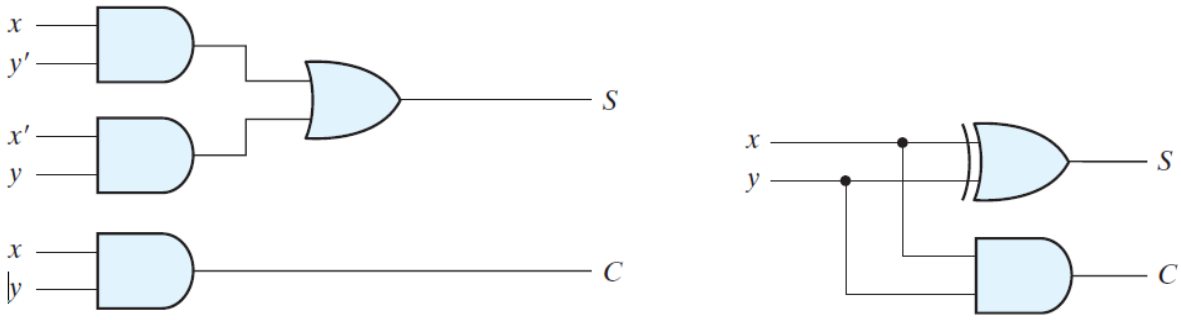
Half Adder is a combinational circuit which adds 2 1-bit digits. Previous carry is not used in the Half Adder. - x, y are the inputs for the addition - S , C = sum & carry of the half adder

$$S = x'y + xy' = x \bigoplus y$$

$$C = xy$$

Table 4.3
Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



(a) $S = xy' + x'y$
$C = xy$

(b) $S = x \oplus y$
$C = xy$

Figure 13: Half Adder

30.2 Full adder

A full adder is a combinational circuits that forms the arithmatic sum of 3 bits (2 significant bits & carry). - x, y are the inputs for the addition - z is the third input which is the carry from the previous lower significant position. - S , C = sum & carry of the full adder

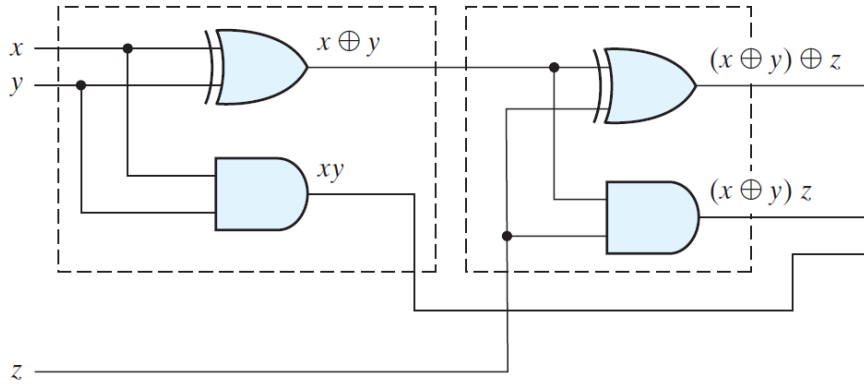
$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

or

$$S = (x \bigoplus y) \bigoplus z$$

$$C = (x \bigoplus y)z + xy$$



Full Adder implementation using Half adder:

30.3 Binary Adder

Binary adder is a digital circuit that produces the arithmatic sum of 2 binary members.

- Binary adder is implemented with full adder connected in cascade.
- Output carry of each full adder is connected to the input carry of the next full adder.
- Addition of n-bit numbers require chain of n-full adders.

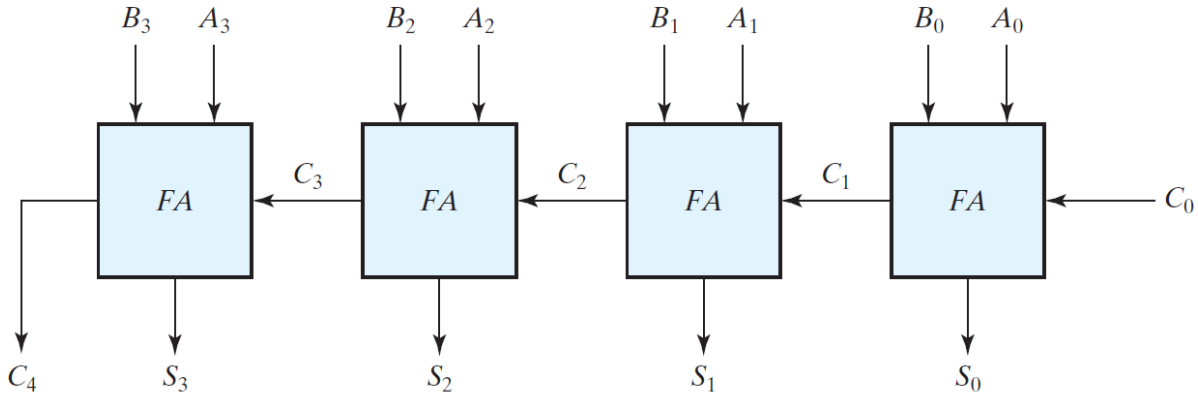


Figure 14: 4 bit Adder with Full adder implementation

This representation is also reffered as 4-bit binary **Ripple Carry adder** as the carry is rippled through the full adders.

For eg. consider, A = 1011, B = 0011 & C_0= 0. The sum and carry is calculated as follows:

Bits	3	2	1	0
C_{in}	0	1	1	0
A_i	1	0	1	1
B_i	0	0	1	1
Sum	1	1	1	0
C_{out}	0	0	1	1

30.3.1 Carry propogation

In the ripple carry adder, to obtain the stable output of the adder, the carry has to be propogated through all the adders, even if all the input bits are available at t=0. - For eg. only after FA_0 calculates C_1 & S_0, FA_1 can start computing the C_2 & S_1 values. All the other full adders will not be giving a valid output. - For an n-bit adder, there are 2n gate levels for the carry to ripple from input to output. - **Carry lookahead logic** is the widely used solution for reducing the carry propogation time in a parallel adder.

In the Carry lookahead logic, the full adder intermediate outputs are named as 2 new binary variables, Carry Propogate (P_i) and Carry Generate (G_i) as shown below:

- $P_i = A_i \bigoplus B_i$
- $G_i = A_i B_i$

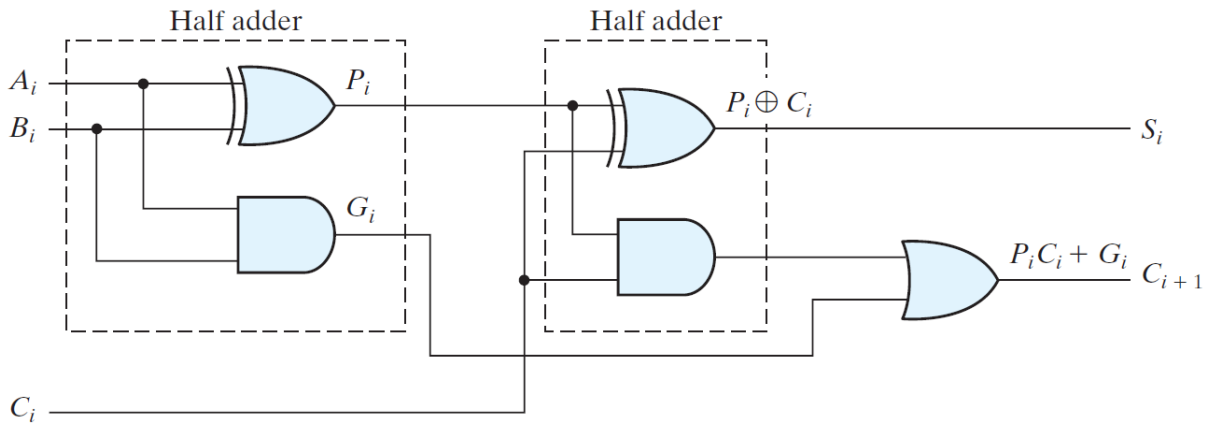


Figure 15: Full adder with Carry Lookahead Logic

With the newly introduced variables sum and carry can be expressed as, - $S_i = P_i \bigoplus C_i$ - $C_{i+1} = G_i + P_i C_i$

Individual carry is calculated and expanded in the primary terms like this: - C_0 = input carry - $C_1 = G_0 + P_0 C_0$ - $C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C0$ - $C_2 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C0$ - $C_{i+1} = G_i + P_i C_i$

With this circuit, the speed of operation is gained at the expense of additional hardware complexity.

Carry Lookahead Generator	4-bit adder with Carry Lookahead
Carry Lookahead Generator	4-bit adder with Carry Lookahead

---

31 Binary Subtractor

Subtraction of binary numbers can be done with the help of compliments. - A - B = A + 2's complement of (B) - For unsigned numbers, - if A $\geqslant$ B
- A - B = A + 2's complement of (B) - if (A < B) - A - B = 2's complement (A + 2's complement of (B)) - For signed numbers, - A - B = A + 2's complement of (B), provided there is no overflow

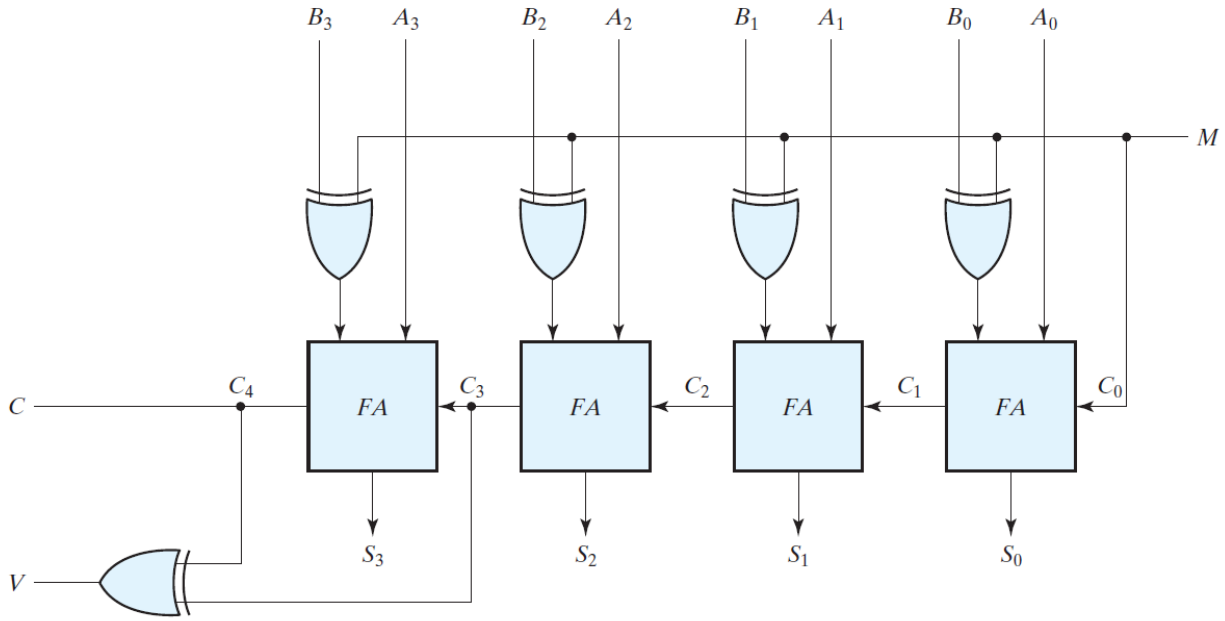


Figure 16: 4-bit Adder Subtractor

Addition and subtraction operations can be combined into one circuit with one common binary adder.

Four bit adder subtractor (with overflow deletion) (mentioned in the image) is implemented as follows: - C_0 is set to 1 for subtraction and 0 for addition - FA_i, adds A_i with 1's compliment of B_i and C_i, which is, - A + 2's complement of (B) for subtraction - A + B for addition - Mode input M controls operation switching - M = 1 for Subtraction - M = 0 for Addition - V = Output for detecting overflow.

31.1 Overflow subtractor

Overflow occurs if 2 'n'-bit numbers are added with 'n+1'-bit result for signed, unsigned, decimal or binary numbers. - Overflow is a problem in digital computers because the number of bits that

hold the number is finite. '(n+1)'-bit result cannot be accommodated by a 'n'-bit word. - Detection of an overflow is critical and is detected with a Flipflop. - Overflow cannot occur after an addition of one positive and one negative number. - An overflow occurs only if both numbers to be added are either both positive or both negative. - An overflow condition can be detected by observing the carry into the sign bit position & the carry out of the sign bit position. - If these 2 carries not equal, an overflow has occurred. - An XOR gate can be implemented for the detection as shown in 4-bit Adder Subtractor.

---

32 Binary Multiplier

- Multiplication in binary form is the same as multiplication in decimal form.
- The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit.
- Each such multiplication forms a partial product. Successive partial products are shifted one position to the left.
- The final product is obtained from the sum of partial products.

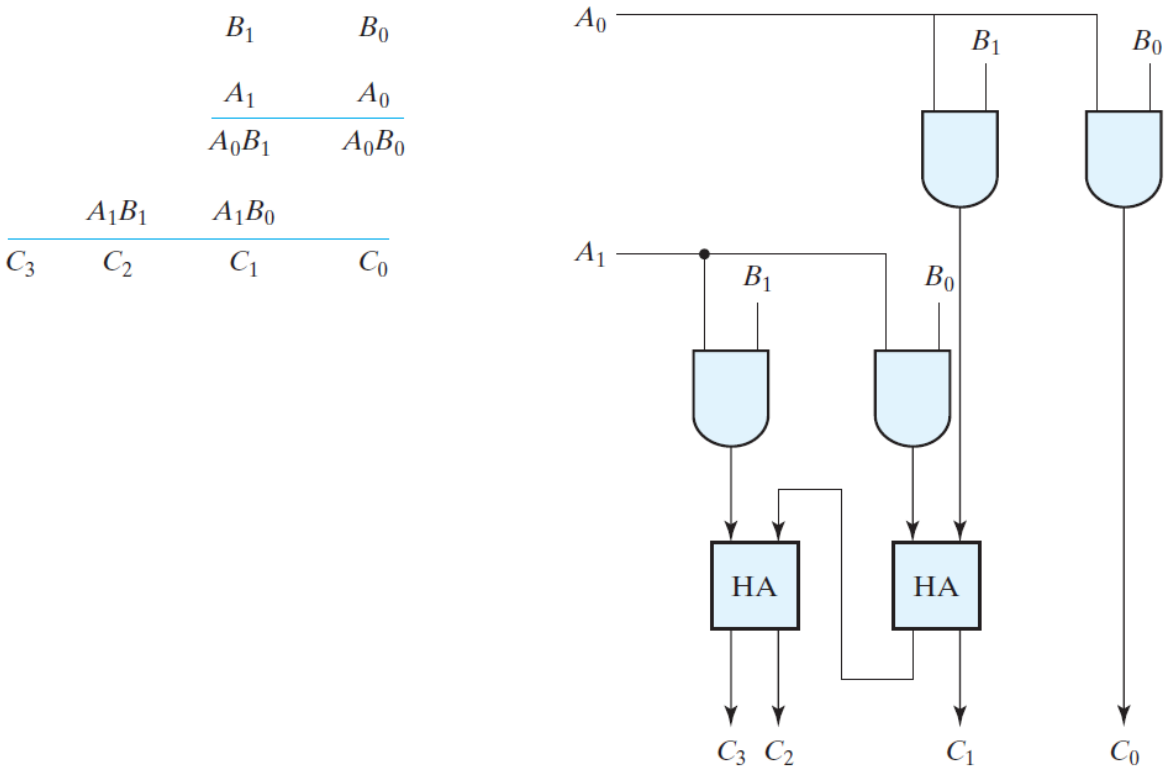


Figure 17: 2-bit by 2-bit binary multuplier

Similarly, for an 'n'-bit multiplication,'n'-bit adders can be implemented instead of Half Adders. For product terms, same AND gate can be used.

---

33 Magnitude Comparator

Magnitude Comparator is a combinational circuit that compares a numbers to determine their relative magnitudes. - For eg. we consider 2 4-bit binary numbers say, - $A = A_3A_2A_1A_0$ - $B = B_3B_2B_1B_0$.

33.1 Test for equality

A & B will be equal if and only if

$$a_0 = b_0, a_1 = b_1, ..., a_n = b_n$$

Actual hardware implementation of the same can be done with the XNOR gate. - $x_i = A_i$ XNOR B_i for all 'n'-bits. - Variable x_i becomes 1 only if both the digits are same.

33.2 Test for comparison

We move from MSB to LSB for the inspection. - $x_i = A_i$ XNOR B_i for all 'n'-bits. - if $x_i = 1$ - i++ until $x_i = 0$ - if $x_i = 0$ - (A > B) if $A_i = 1$, $B_i = 0$ - (A < B) if $A_i = 0$, $B_i = 1$

33.3 Boolean functions for Magnitude Comparator

$$(A > B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$$
$$(A < B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$$
$$(A = B) = x_3x_2x_1x_0$$

These expressions can be implemented on hardware with basic logic gates.

---

34 Decoders

A Decoder is a combinational circuit that converts binary information from 'n' input lines to a maximum of 'm' unique output lines. - The decoders that convert n-line inputs to m-line outputs are called n to m line decoders where, $m \leqslant 2^n$. - Ex. **3 to 8 line decoder**: 3 inputs are decoded into 8 outputs. - The input variables represent a binary number & the outputs represent all the combinations of that binary number. - Typical Application: binary to octal conversion.

3x8 Decoder

Figure 18: 3x8 Decoder

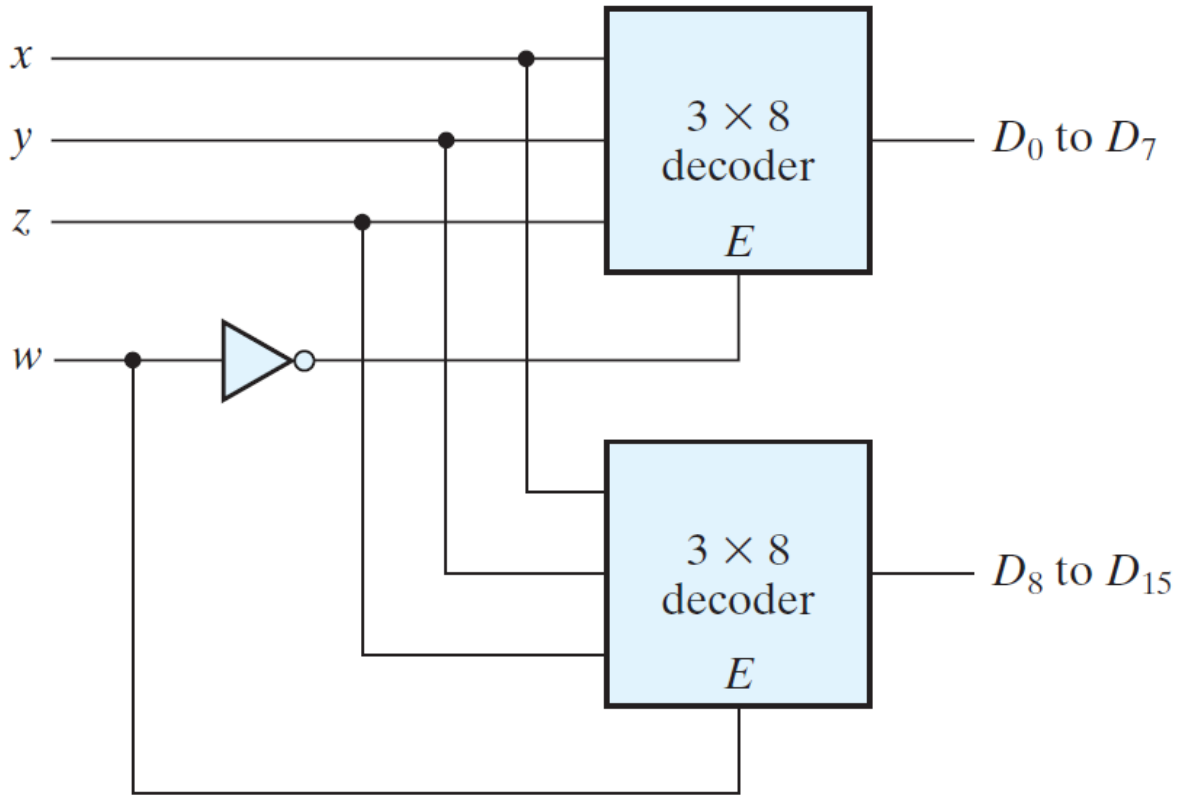
34.1 Decoder Applications

34.1.1 Decoder Demultiplexer

A decoder with enable input can function as a demultiplexer. Enable line being the input and other lines as select lines.

34.1.2 Nested Decoders

Decoder with enable inputs can be connected together to form a larger decoder circuits.



34.1.3 Combinational Logic

Any combinational circuit with 'n' inputs and 'm' outputs can be implemented with an 'n' to '2^n' line decoder and 'm' OR gates. For eg. Implementation of a full adder with a decoder.

---

35 Encoder

An encoder generates the binary code corresponding to each input value. Encoder has 2^n (or fewer) input lines and 'n' output lines.

35.1 Priority encoder

A priority encoder is an encoder that handles inputs with certain predefined priority, to generate output accordingly. For eg. this is a 4-bit priority encoder with the following truth table.

D0	D1	D2	D3	a	b	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1

D0	D1	D2	D3	a	b	V
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

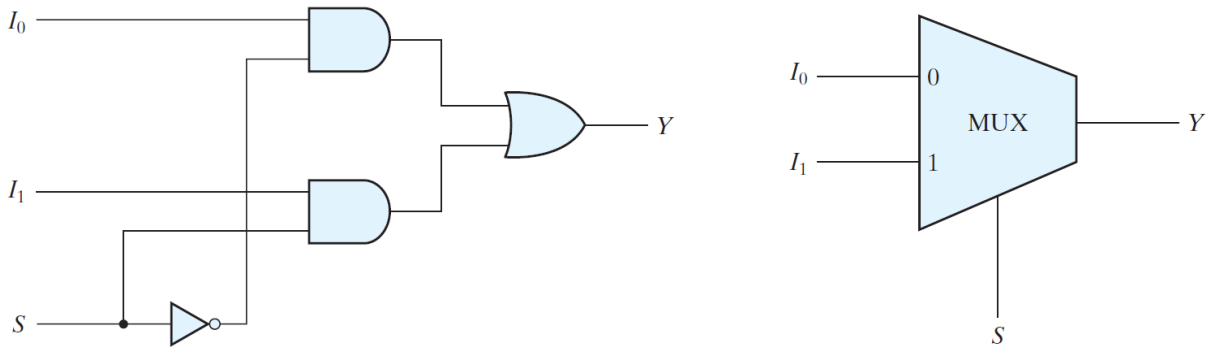
- D0 - D3 = Inputs
- a, b, V = Outputs
- X = Don't care
- V = valid bit indicator
  - V is set when 1 or more inputs are equal to 1
  - when all inputs are 0, there is no valid inout and V = 0
- High Priority for higher subscript number i.e. D3 has more priority than D2

---

36 Multiplexer

A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line based on select lines.

- If there are 'n'select lines, the input lines should be'2^n'.



(a) Logic diagram (b) Block diagram

Figure 19: 2x1 Multiplexer

Boolean expression can also be implemented with MUX.

MUXs can be combined with common select lines to provide multiple bit selection logic. For eg. Quadrapule 2:1 line Mux below uses same select line to select either A or B channel.

Quadrapule 2:1 line Mux

Figure 20: Quadrapule 2:1 line Mux

36.1 Three state gate (Tristate gate) (Tristate buffer)

MUX can be constructed with Three State Gates (digital circuits that exhibit 3 States) These 3 states include :

1. Logic 1
2. Logic 0
3. High impedance state

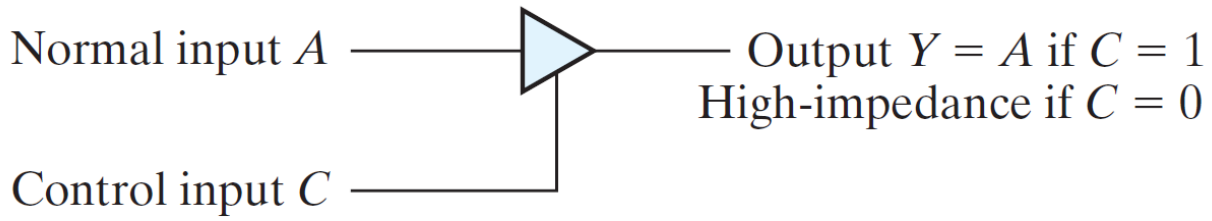


Figure 21: Three state gate

36.1.1 High impedence state

- Logic behaves like an open circuit.
- The circuit has no logic significance.
- The circuit connected to the output of the 3 state gate is not affected by the inputs to the gate.
- Tristate gates may perform any conventional logic.
- However, the most commonly used tristate gate is the buffer gate.
- For eg. Implementation of 2:1 Mux with tristate gates.

Implementation of Mux with tristate gates

Figure 22: Implementation of Mux with tristate gates

---

37 HDL models of combinational circuits

Revisit

38 Synchronous Sequential Logic

---

39 Introduction

A sequential circuit is specified by a time sequence of inputs, outputs, and internal states. It consists of a combinational circuit to which memory elements are connected to form a feedback path.

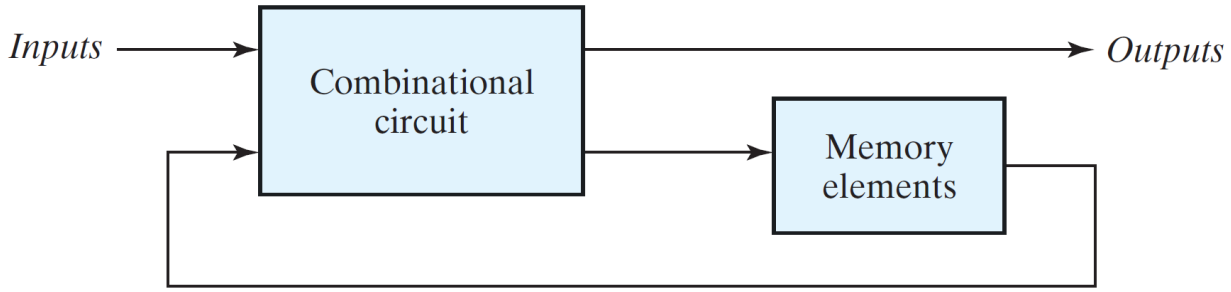


Figure 23: Block diagram of sequential circuit

- The storage elements are devices capable of storing binary information.
- The binary information stored in the memory elements determines the "state" of the sequential circuit.
- The sequential circuits receive binary information from the external inputs that, together with the present state of the storage elements, determine the binary value of the outputs.
- The next state of the storage elements is also a function of external inputs and the present state.

---

40 Types of sequential circuits

- **Synchronous sequential circuits**: Circuits whose behavior can be defined from the knowledge of it's signals at discrete instances of time.
- **Asynchronous sequential circuits**: Circuits whose behavior depends on the input signals at any instance of time and the order in which the input changes.

In synchronous sequential circuits, synchronization is achieved by a timing device called as *clock generator*, which provides a periodic signal in the form of train of *clock pulses*. The clock signal is commonly denoted by the identifiers ***clock*** or ***clk***. The clock pulses are distributed throughout the system.

- A clock pulse goes through two transitions:
  - from 0 to 1
  - from 1 to 0.
- The 0 to 1 transition is defined as the positive edge and the 1 to 0 transition as the negative edge.

Synchronous sequential circuits, that use clock pulses to control storage elements are called *clocked sequential circuits*. The storage elements (memory) used in clocked sequential circuits are called Flip-Flops. A Flip-Flop is a binary storage device capable of storing one bit of information.

A storage element in a digital circuit can maintain a binary state indefinitely (as long as power is delivered to the circuit), until directed by an input signal to switch states.

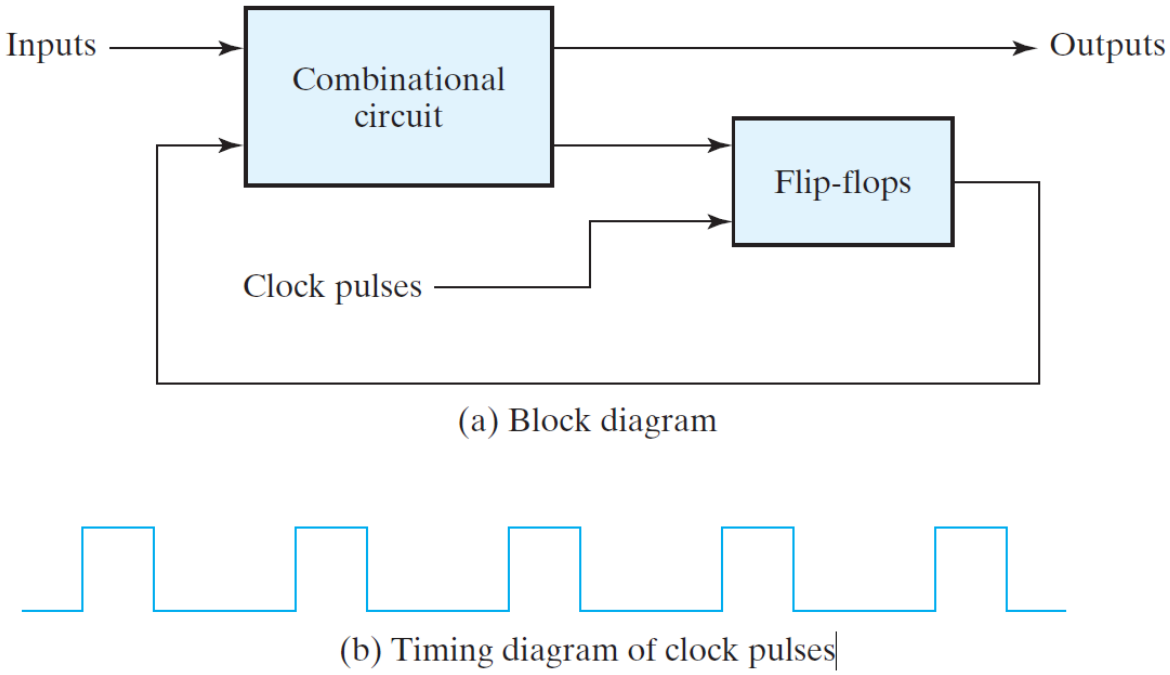


Figure 24: Synchronous Sequential circuit

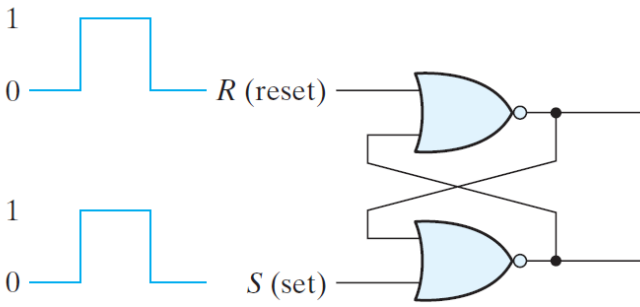
41 Storage Elements: Latches

Storage elements that operate with signal levels (rather than signal transitions) are referred to as ***latches*** & those controlled by a clock transition are ***flip-flops***. - Latches are said to be level sensitive devices; flip-flops are edge-sensitive devices. - The two types of storage elements are related because latches are the basic circuits from which all flip-flops are constructed.

41.1 SR Latch (Set Reset Latch)

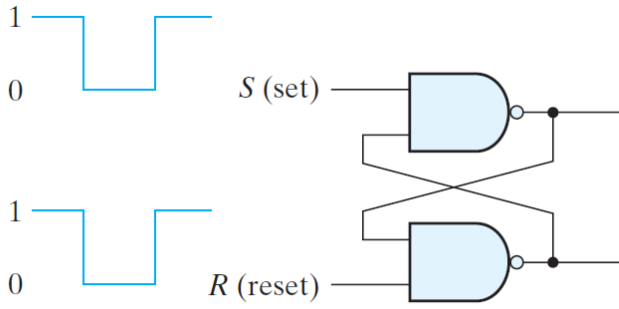
The SR latch is a circuit with either two cross-coupled NOR gates or two cross-coupled NAND gates, and two inputs labeled for set(S) and reset(R). - The SR latch has two useful states: - **Set state:** When output is tied high, the latch is said to be in the set state. - **Reset state:** When output is tied low, the latch is said to be in the reset state. - Outputs Q and Q' are normally the complement of each other.

- Setting both inputs to 1 is forbidden as it might lead to unpredictable/ "metastable" state in a SR Latch with NOR gates.
- When both inputs are equal to 1 at the same time, a condition in which both outputs are equal to 0 (rather than be mutually complementary) occurs.



(a) Logic diagram

SR Latch implementation with two cross-coupled NOR gates:



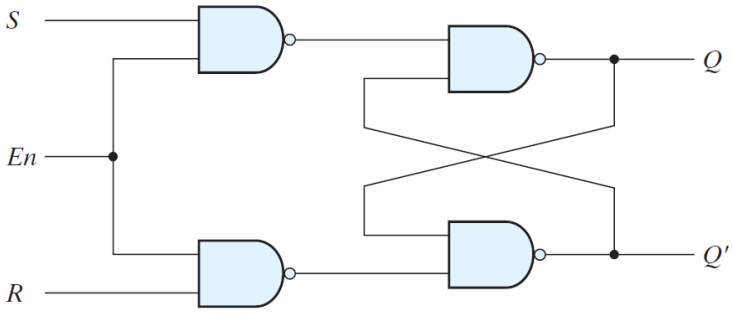
(a) Logic diagram

SR Latch implementation with two cross-coupled NAND gates:

- In SR Latch witn NAND implementation, (0,0) is the forbidden output for the same reason.

41.2 SR latch with control input

The operation of the basic SR latch can be modified with an additional control input signal **"*En*"**. - Only when *"En"* goes low, information from the S or R input is allowed to affect the latch. - The control input *"En"* acts as an enable signal for the other two inputs. - *"En"* in turn, controls the state of the latch as well. - An indeterminate condition occurs when all three inputs are equal to 1.



(a) Logic diagram

En	*S*	*R*	Next s
0	X	X	No ch
1	0	0	No ch
1	0	1	$Q = 0$
1	1	0	$Q = 1$
1	1	1	Indete

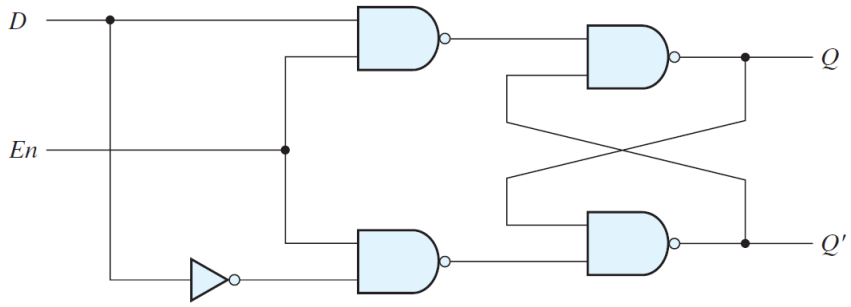
(b) Function

SR latch with control input:

41.3 D (Transparent) latch

To eliminate the metastable condition of the SR latch, the inputs S and R should never be high at the same time. This is done in the D latch.

- D latch has only two inputs: D (data) and En (enable).
- D signal goes to the S input.
- D complement goes to the R input.
- The output follows changes in the data input as long as the enable input is asserted.



(a) Logic diagram

En	*D*	Next state of *Q*
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

(b) Function table

Figure 25: D Latch

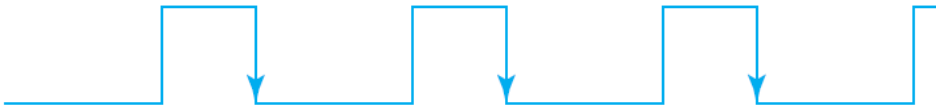
---

42 Storage Elements: flip-flops

- In sequential circuit, outputs are connected to the inputs of the storage element through the combinational circuit.
- The state transitions of the latches start as soon as the control signal becomes high.
- If the inputs of the latches change while the control signal is high, the latches will respond to new values and a new output state may occur.
- Hence the state of the latches may keep changing for as long as the control signal is high.

(a) Response to positive level

(b) Positive-edge response



(c) Negative-edge response

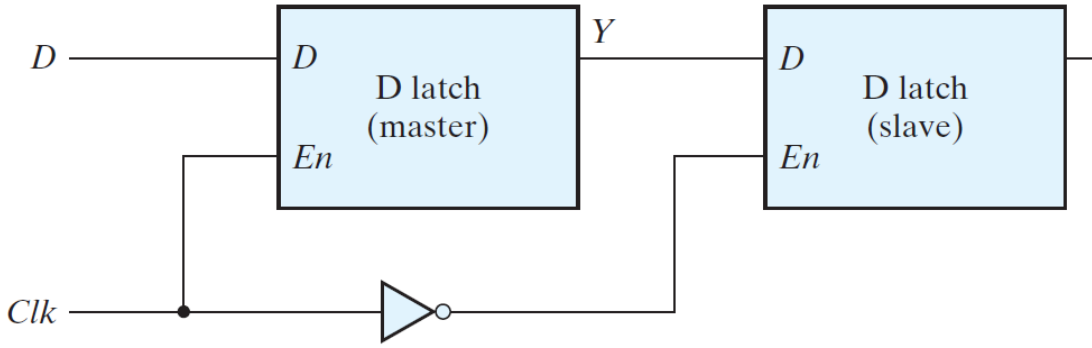
Clock response in latch and flip-flop:

- The latch responds to a change in the level of a clock pulse whereas, a flip-flop triggers only during a signal transition.
- There are two ways to transform a latch into a flip-flop:
  1. Employ two latches in a special configuration that isolates the output from the input changes.
  2. A latch that triggers only during both transitions of the control signal and is disabled during the rest of the clock pulse.

  - In flip-flops, the control signal is a continuous train of pulses and is often referred to as **clk**.
  - In VLSI, the most economical and efficient flip-flop constructed is the edge-triggered D flipflop, as it uses the smallest number of gates.
  - Two flip-flops less widely used in the design of digital systems are the JK and T flip-flops.

42.1 Edge-Triggered D Flip-Flop

A D flip-flop is constructed with two D latches and an inverter. - The first latch is called the master and the second the slave. - The circuit samples the D input and changes its output Q only at the negative edge of the **clk**.

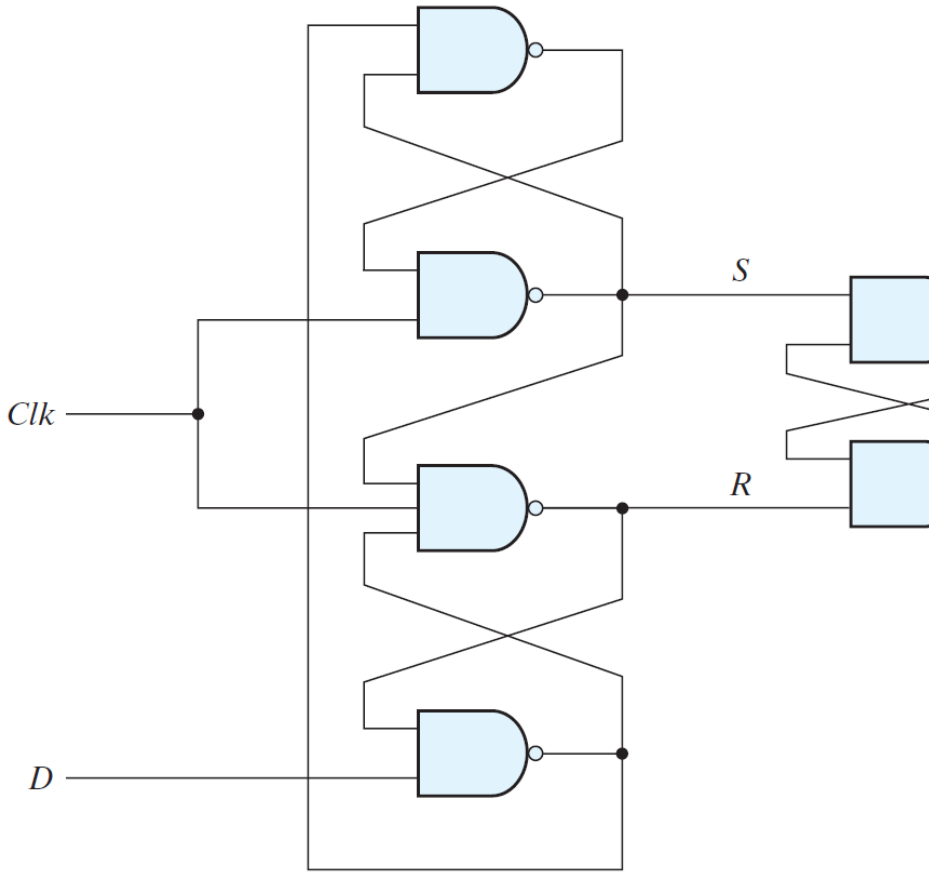


Master–slave D flip-flop:

- Configuration 1:
  - Clk = 0, clk'(slave clk) = 1
  - Slave latch is enabled & master latch is disabled
  - Y –> Q
  - Y will not change after 1 to 0 clock transition
  - Hence, Q will also be constant for the whole pulse.
- Configuration 2:
  - Clk = 1, clk'(slave clk) = 0
  - Slave latch is disabled & master latch is enabled
  - D –> Y
  - D will not change after 0 to 1 clock transition
  - Hence, Y will also be constant for the whole pulse.

  Effectively, as disabled latches act as a buffer and a change in the output of the flip-flop can be triggered only by and during the transition of the clock from 1 to 0.

Another construction of an edge-triggered D flip-flop uses three SR latches as shown below:



D -type positive-edge-triggered flip-flop

- Two latches respond to the external D (data) and Clk (clock) inputs.
- The third latch provides the outputs for the flip-flop.

42.1.1 Flip-flop timing

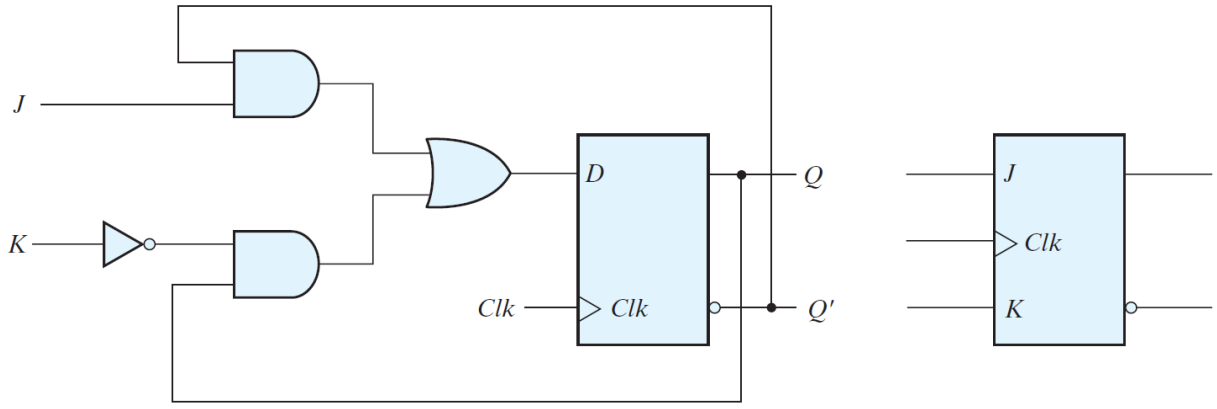
- Minimum time during which the input must not change prior to the clock transition is called as **setup time.**
- Minimum time during which the input must not change after the clock transition is called as **hold time.**
- The propagation delay time of the flip-flop is defined as the interval between the trigger edge and the stabilization of the output to a new state.

42.2 JK flip-flop

There are three operations that can be performed with a flip-flop: 1. Set 2. Reset 3. Complement its output.

Synchronized by a clock signal, the JK flip-flop has two inputs and performs all three operations. 1. J = 1, K = 0, sets the flip-flop 2. J = 0, K = 1, resets the flip-flop 3. J = K = 1, the output is complemented. 3. J = K = 0, the output is unchanged.

JK flip-flop can be constructed with a D flip-flop and gates: > D = JQ' + K'Q



(a) Circuit diagram (b) Graphic symbol

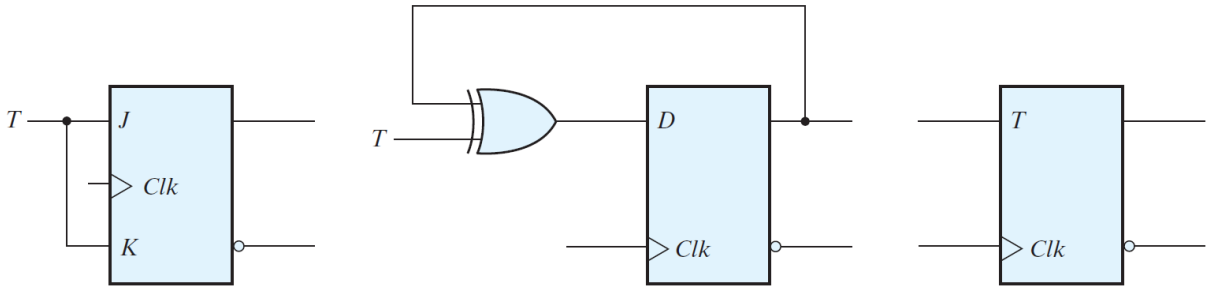
JK flip-flop:

42.3 T flip-flop

The T (toggle) flip-flop is a complementing flip-flop and can be obtained from a JK flip-flop when inputs J and K are tied together.

1. T = 1 (J = K = 1), the output is complemented.
2. T = 0 (J = K = 0), the output is unchanged.

The T flip-flop can be constructed with a D flip-flop and an XOR gate: > D = T⊕Q = TQ' + T'Q



(a) From *JK* flip-flop (b) From *D* flip-flop (c) Graphic symbol

T flip-flop:

42.4 Characteristic Tables

- A characteristic table defines the logical properties of a flip-flop by describing its operation in tabular form.
- They define the next state $Q(t+1)$ as a function of the inputs $(D/T/J, K)$ and the present state $Q(t)$.

42.5 Characteristic Equations

The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation.

1. D flip-flop: $Q(t+1) = D$
2. JK flip-flop: $Q(t+1) = JQ' + K'Q$

JK Flip-Flop

J	K	Q(t + 1)	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

D Flip-Flop

D	Q(t + 1)	
0	0	Reset
1	1	Set

T Flip-Flop

T	Q(t + 1)	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

Figure 26: Flip-Flop Characteristic Tables

3. T flip-flop: $Q(t + 1) = T

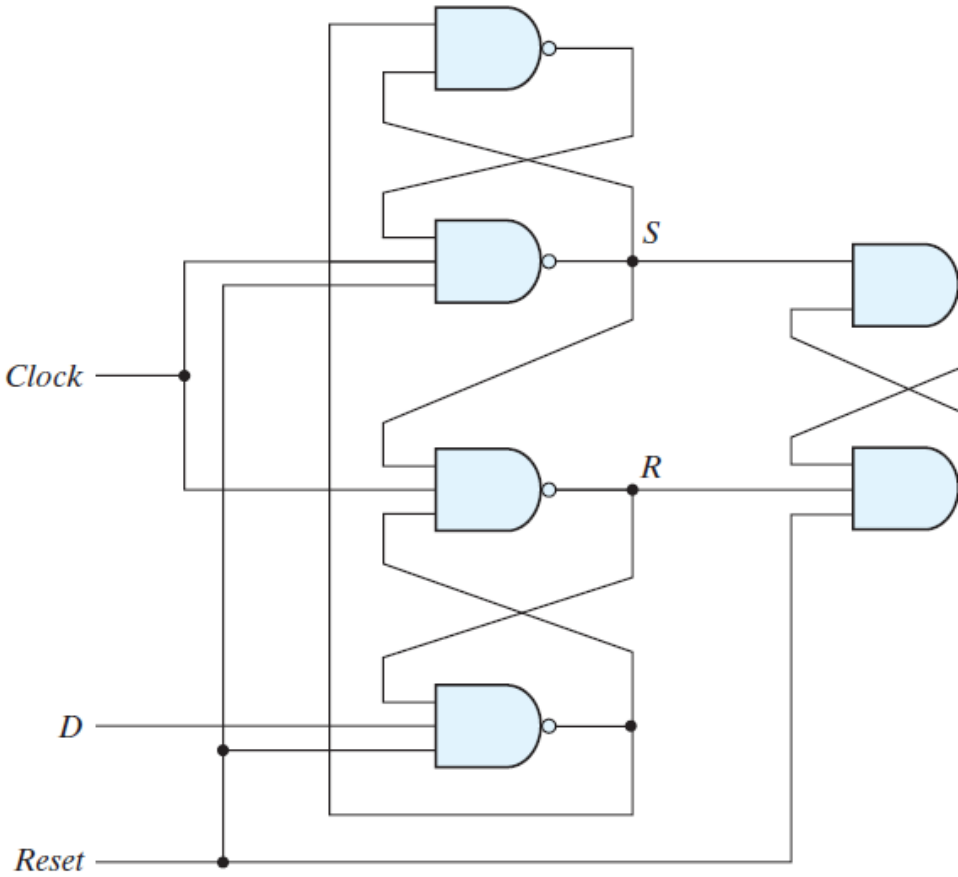
$$\oplus$$

Q = TQ' + T'Q$

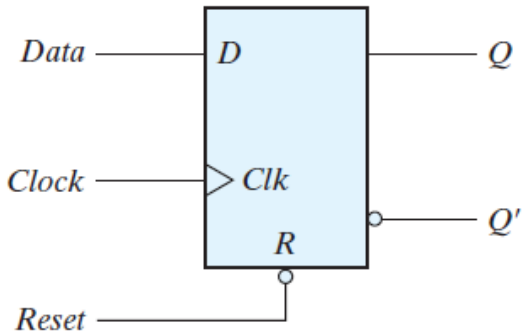
42.6 Direct Inputs

Some flip-flops have asynchronous inputs that are used to force the flip-flop to a particular state independently of the clock.

- The input that sets the flip-flop to 1 is called **preset** or **direct set**.
- The input that clears the flip-flop to 0 is called **clear** or **direct reset**.



(a) Circuit diagram



(b) Graphic symbol

R	Clk	D
0	X	X
0	↑	0
0	↑	1

(b) Functi

D flip-flop with asynchronous reset:

---

43 Analysis of Clocked Sequential Circuits

- The behavior of a clocked sequential circuit is determined from the inputs, the outputs, and the state of its flip-flops.
- The outputs and the next state are both a function of the inputs and the present state.

- The analysis of a sequential circuit consists of obtaining a table or a diagram for the time sequence of inputs, outputs, and internal states.
- A state table and state diagram are then presented to describe the behavior of the sequential circuit.

43.1 State Equations

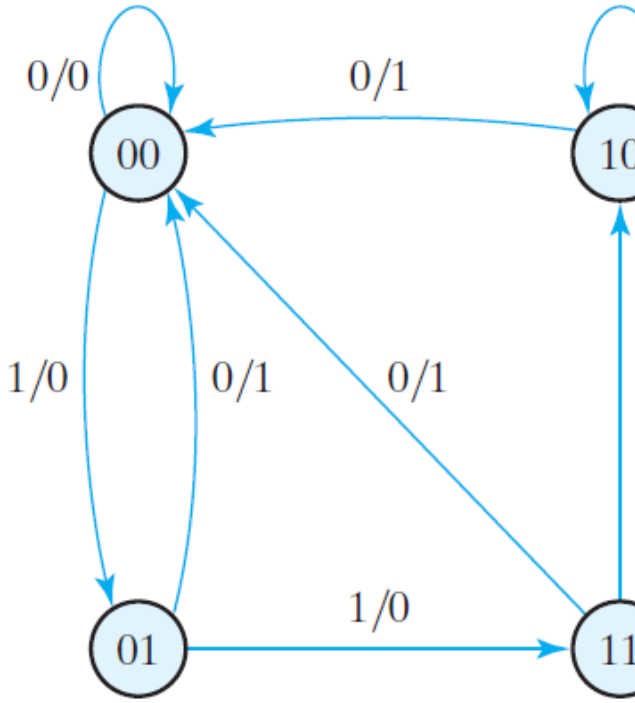
A **state equation** (or *transition equation*) specifies the next state as a function of the present state and inputs.

43.2 State Table

The time sequence of inputs, outputs, and flip-flop states can be enumerated in a **state table** (or *transition table*).

- The table consists of four sections labeled present state, input, next state, and output.
  - present state: states of flip-flops at any given time t
  - input: value of input signal for each possible present state
  - next state: states of the flip-flops one clock cycle later $(t + 1)$
  - output: value of output signal at time t for each present state and input condition.
- The derivation of a state table requires listing all possible binary combinations of present states and inputs.
- The next-state values are then determined from the logic diagram or from the state equations.

Present State		Next State				x
		x = 0		x = 1		
A	B	A	B	A	B	
0	0	0	0	0	1	
0	1	0	0	1	1	
1	0	0	0	1	0	
1	1	0	0	1	0	



A sample State Table & State diagram:

43.3 State Diagram

The information available in a state table can be represented graphically in the form of a state diagram.

- State is represented by a circle
- The binary number inside each circle identifies the state of the flip-flops.

- Transitions of states are indicated by directed lines connecting states
- The directed lines are labeled with two binary numbers separated by a slash.
  - Number before the slash: input during the present state
  - Number after the slash: output during the present state with the given input.

The following steps are followed during the analysis of clocked sequential circuits:

Circuit diagram → Equations → State table → State diagram

This sequence of steps begins with a structural representation of the circuit and proceeds to an abstract representation of its behavior.

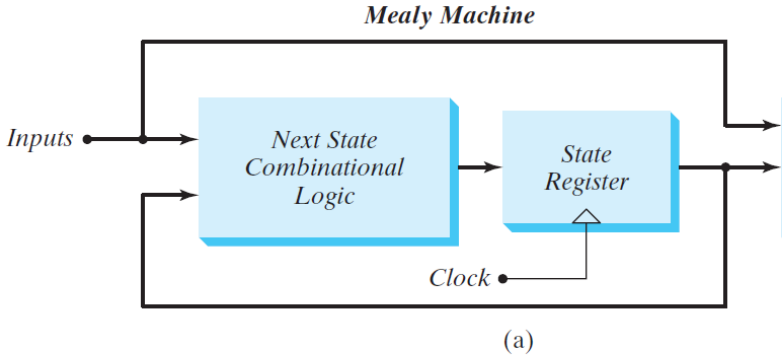
43.4 Flip-Flop Input Equations

- The logic diagram of a sequential circuit consists of flip-flops and gates.
- Logic diagram of the sequential circuit requires:
  - type of flip-flops
  - list of the Boolean expressions of the combinational circuit
- The part of the combinational circuit that generates external outputs is described algebraically by a set of Boolean functions called **output equations**.
- The part of the circuit that generates the inputs to flip-flops is described algebraically by a set of Boolean functions called flip-flop **input equations** (or *excitation equations*).

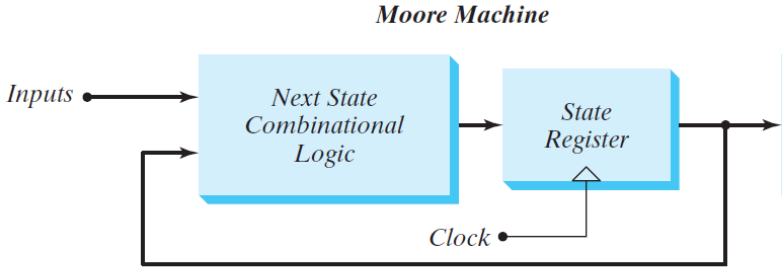
43.5 Mealy and Moore Models of Finite State Machines

The most general model of a sequential circuit has inputs, outputs, and internal states.

There are two models of representing sequential circuits: 1. Mealy model 2. Moore model



(a)



(b)

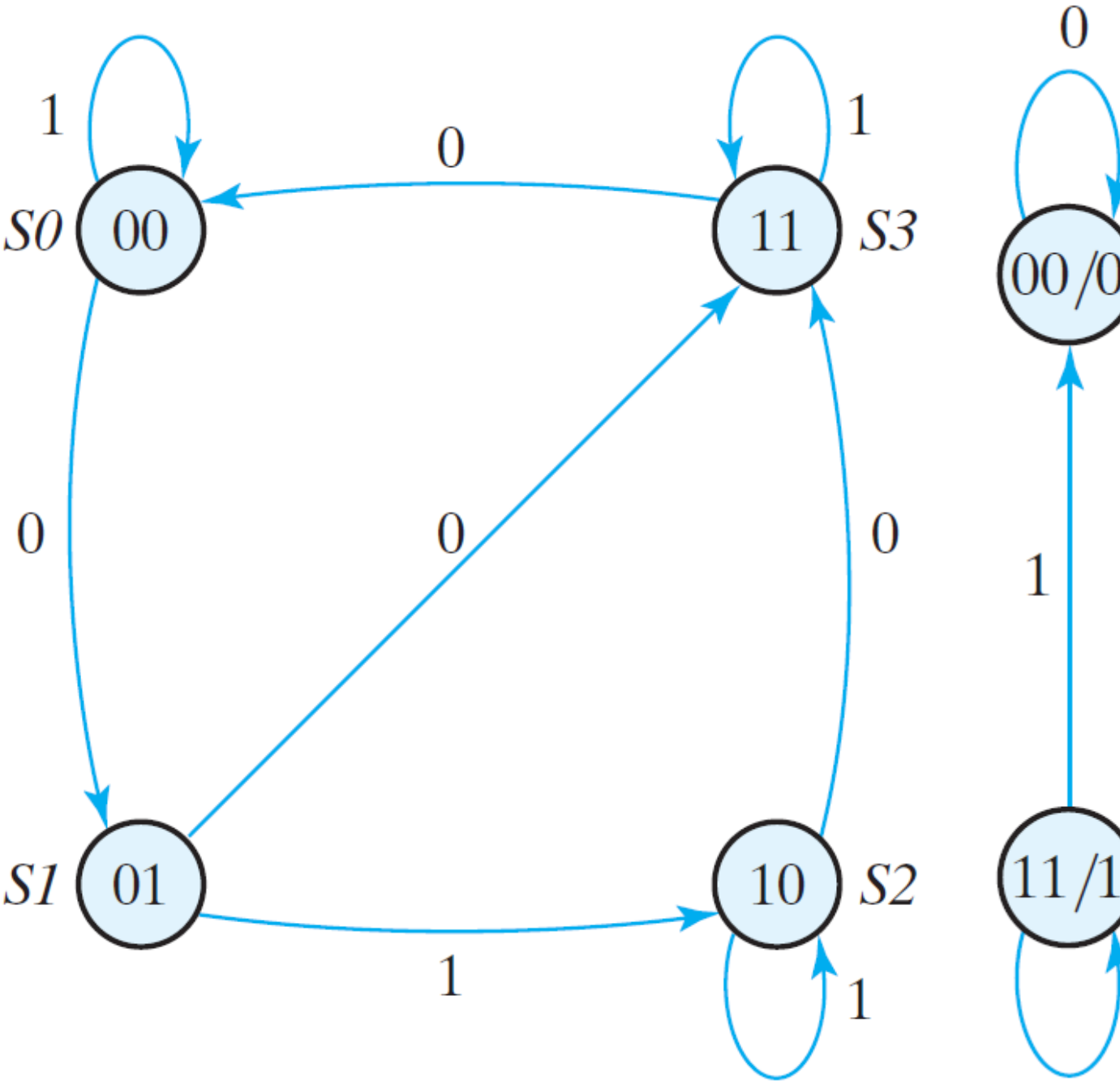
Block diagrams of Mealy and Moore state machines:

- These two models of a sequential circuit are commonly referred to as a ***Finite State Machine***, abbreviated **FSM**.

  They differ only in the way the output is generated.

Examples of Mealy & Moore State Machines:

Mealy State Machine



43.5.1 Mealy model

- In mealy model, the output is a function of both the present state and the input.
- The Mealy model of a sequential circuit is referred to as a *Mealy FSM* or *Mealy machine*.
- the output of the Mealy machine is the value that is present immediately before the active edge of the clock.

43.5.2 Moore model

- In the Moore model, the output is a function of only the present state. A circuit may have both types of outputs.
- The Moore model is referred to as a *Moore FSM* or *Moore machine*.
- The outputs of the sequential circuit are synchronized with the clock, because they depend only on flip-flop outputs that are synchronized with the clock.

---

44 State Reduction and Assignment

- The *analysis* of sequential circuits starts from a circuit diagram and culminates in a state table or diagram.
- The *design* (synthesis) of a sequential circuit starts from a set of specifications and culminates in a logic diagram.
- The goal here is to simplify a design by reducing the number of gates and flip-flops it uses.

44.1 State Reduction

- The reduction in the number of flip-flops in a sequential circuit is referred to as the *state-reduction* problem.
- It is more convenient to apply procedures for state reduction with the use of a table rather than a diagram.
- Algorithm for the state reduction of a completely specified state table: > Two states are said to be equivalent if, for each member of the set of inputs, they give exactly the same output and send the circuit either to the same state or to an equivalent state.
- When two states are equivalent, one of them can be removed without altering the input–output relationships.

44.2 State Assignment

- In order to design a sequential circuit with physical components, it is necessary to assign unique coded binary values to the states.
- For a circuit with 'm' states, the codes must contain 'n' bits, where $2n \geq m$. For example, n = 3, codes = 000 - 111.
- Unused states are treated as don't-care conditions during the design.
- Types of state assignments:
  - binary numbers
  - decimal equivalents
  - Gray code

  - one-hot assignment
- The deciding factors of the assignments:
  - simplicity of the design
  - decoding logic
  - silicon area
  - speed
  - efficiency

---

45 Design Procedure

- Design procedures or methodologies specify hardware that will implement a desired behavior.
- The design effort for small circuits may be manual, but industry relies on automated synthesis tools for designing massive integrated circuits.
- The sequential building block used by synthesis tools is the D flip-flop.
- Together with additional logic, it can implement the behavior of JK and T flip-flops.
- The first step in the design of sequential circuits is to obtain a state table or a state diagram.
- The number of flip-flops is determined from the number of states needed in the circuit and the choice of state assignment codes.
- The combinational circuit is derived from the state table by evaluating the flip-flop input equations and output equations.

The procedure for designing synchronous sequential circuits comprises of the following steps: 1. From the word description and specifications of the desired operation, derive a state diagram for the circuit. 2. Reduce the number of states if necessary. 3. Assign binary values to the states. 4. Obtain the binary-coded state table. 5. Choose the type of flip-flops to be used. 6. Derive the simplified flip-flop input equations and output equations. 7. Draw the logic diagram.

- The part of the design that follows a well-defined procedure is referred to as synthesis.
- Designers using logic synthesis tools (software) can follow a simplified process that develops an HDL description directly from a state diagram, letting the synthesis tool determine the circuit elements and structure that implement the description.

---

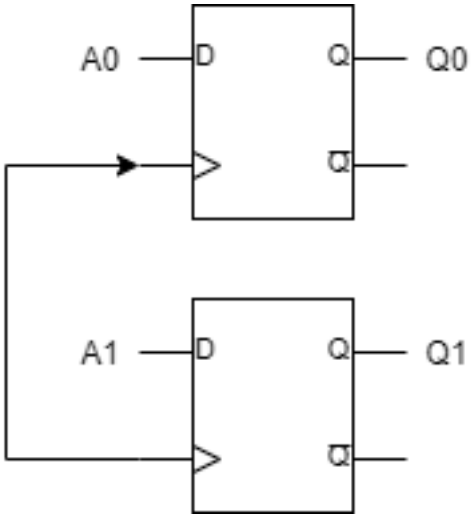
46 Registers

- A circuit with flip-flops is considered a sequential circuit even in the absence of combinational gates.
- In the absence of flip-flops, the circuit reduces to a purely combinational circuit.
- Sequential circuit are usually classified by the function they perform. Two such circuits are:
  1. *Registers*:
     - A *register* is a group of flip-flops, each one of which shares a common clock and is capable of storing one bit of information.
     - An n-bit register consists of a group of n flip-flops capable of storing n bits of binary information.

2. *Counters*:
   - A *counter* is a register that goes through a predetermined sequence of binary states.

- The simplest register is one that consists of only flip-flops, without any gates.
- The common clock triggers all flip-flops on the positive edge of each pulse, and the binary data available at the four inputs are transferred into the register.

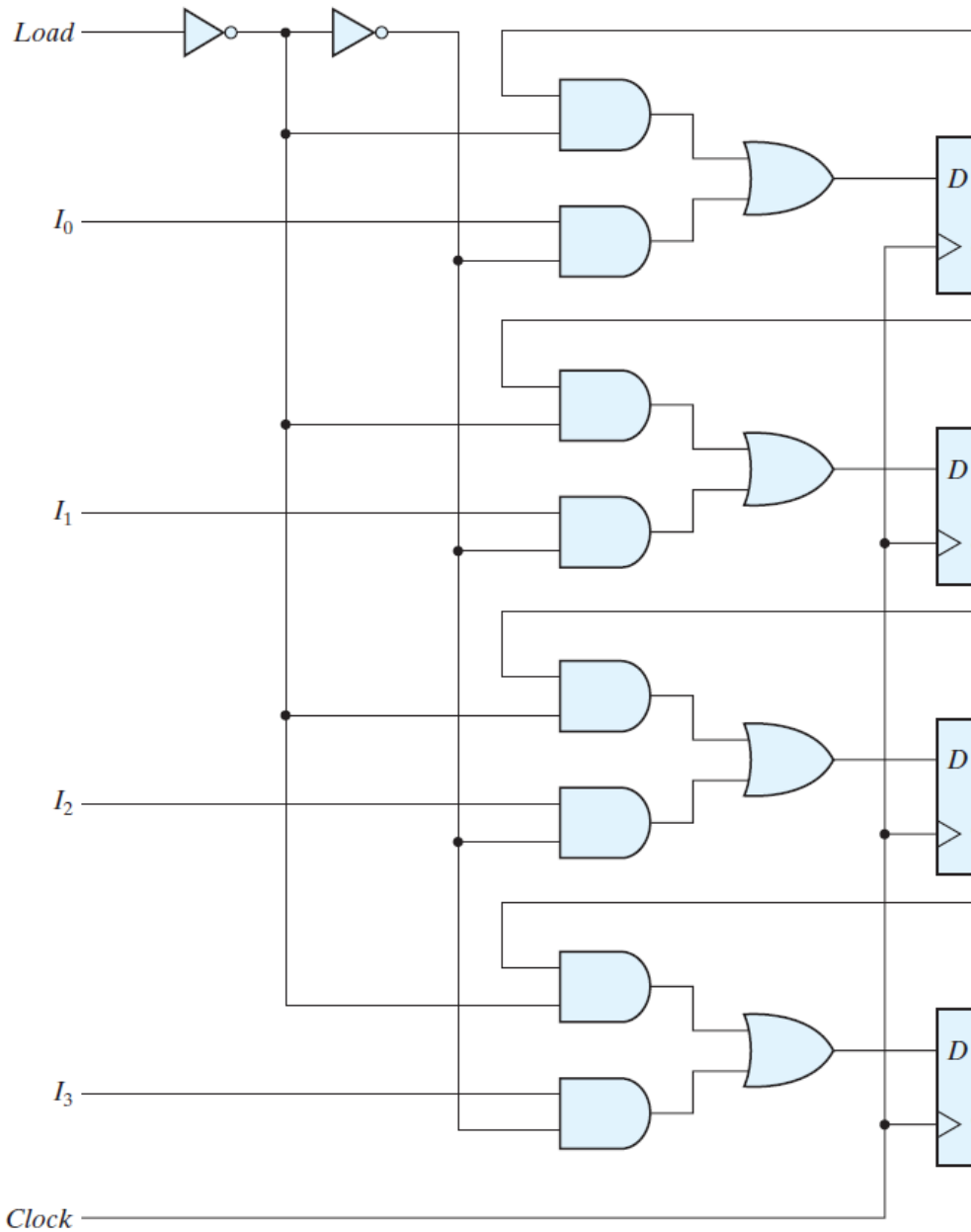
A simple two-bit register using D flip-flops:



46.1 Register with Parallel Load

- Registers with parallel load are a fundamental building block in digital systems.
- Synchronous digital systems have a master clock generator that supplies a continuous train of clock pulses.
- The transfer of new information into a register is referred to as *loading* or *updating* the register.
- If all the bits of the register are loaded simultaneously with a common clock pulse, the loading is known as *parallel*.
- In a parallel register loading, to hold the contents of the register:
  1. The inputs must be held constant or
  2. The clock must be inhibited from the circuit
- In both the cases, data or clock will be unavailable for other registers/ rest of the circuit.
- Gates are not inserted into the clock path because it produces uneven propagation delays between the master clock and the inputs of flip-flops.
- To fully synchronize the system, we must ensure that all clock pulses arrive at the same time anywhere in the system, so that all flip-flops trigger simultaneously.

  Hence, D inputs are used to control the operation of the register.



Four-bit register with parallel load:

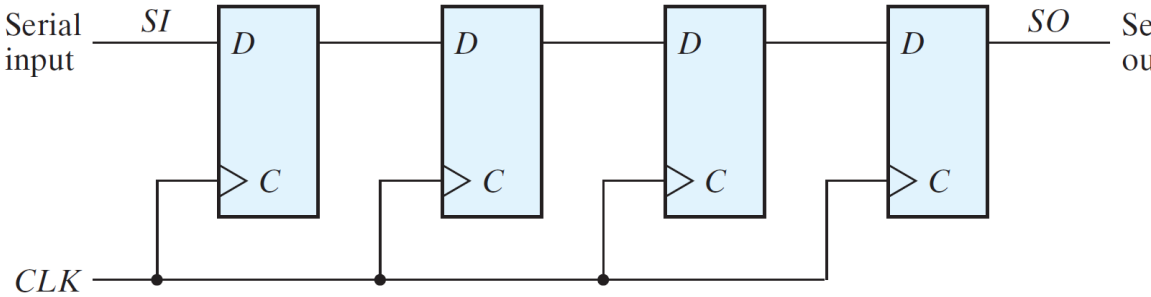
- The additional gates implement a two-channel mux whose output drives the input to the register with either the data bus or the output of the register.

---

47 Shift Registers

A register capable of shifting the binary information held in each cell to its neighboring cell, in a selected direction, is called a *shift register*.

- The logical configuration of a shift register consists of a chain of flip-flops in cascade.
- The simplest possible shift register uses only flip-flops, as shown below:



Four-bit shift register

- Each clock pulse shifts the contents of the register one bit position to the right.
- The configuration does not support a left shift and is unidirectional.

47.1 Serial Transfer

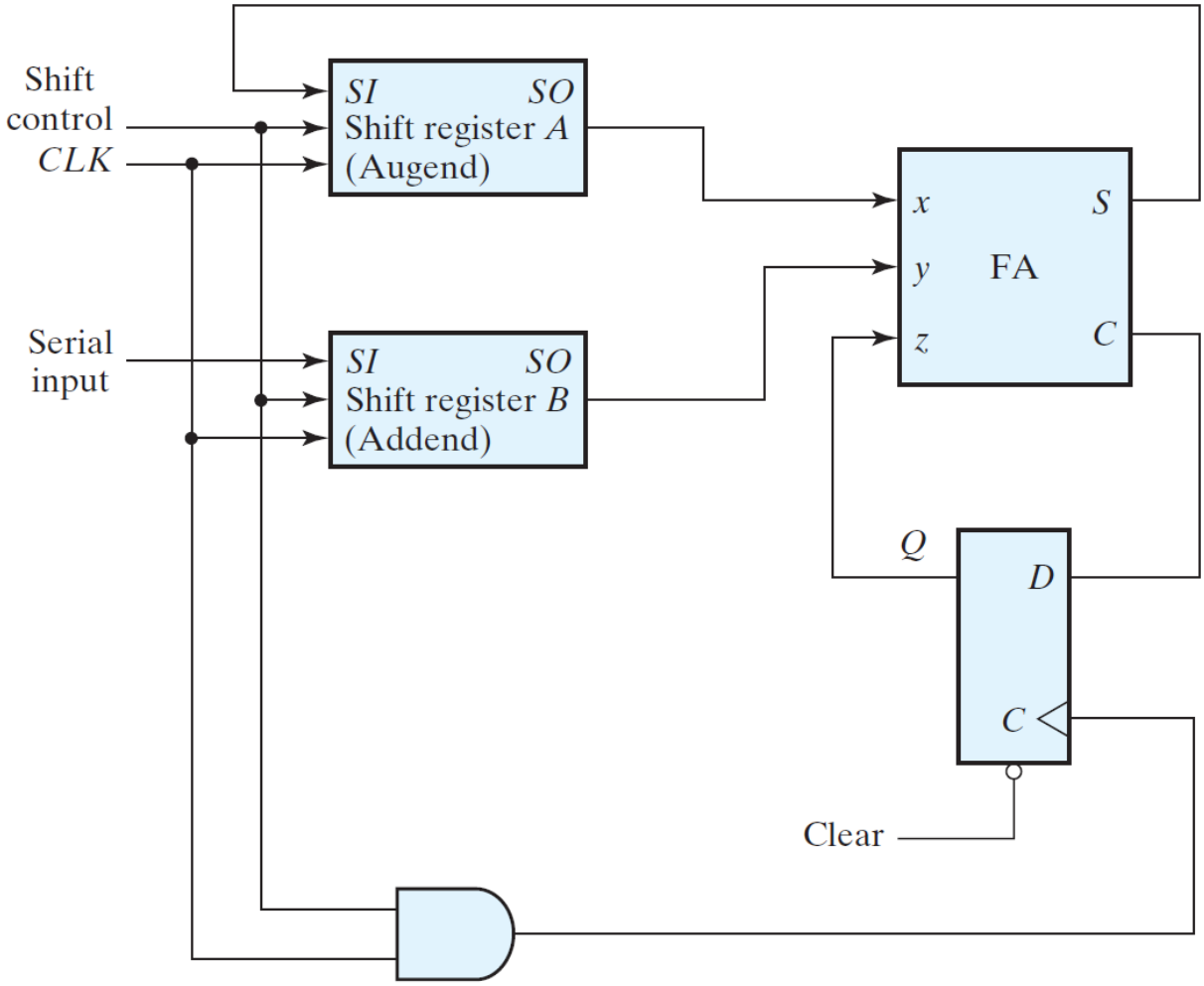
- The datapath of a digital system is said to operate in serial mode when information is transferred and manipulated one bit at a time.
- In the parallel mode, information is available from all bits of a register and all bits can be transferred simultaneously during one clock pulse.
- In the serial mode, the registers have a single serial input and a single serial output. The information is transferred one bit at a time while the registers are shifted in the same direction.
- The initial content of a register is shifted out through its serial output and is lost unless it is transferred to another shift register.

47.2 Serial Addition

- Operations in digital computers are usually done in parallel because that is a faster mode of operation.
- Serial operations are slower because a datapath operation takes several clock cycles
- Serial operations require fewer resources in hardware.

47.2.1 Serial Adder

- The two binary numbers to be added serially are stored in two shift registers.
- The addition is accomplished by passing each pair of bits together with the previous carry through a single full-adder circuit and transferring the sum, one bit at a time, into register A.
- The sum bit from the S output of the full adder could be transferred into a third shift register.
- It is possible to use one register for storing both the augend and the sum bits.
- The carry out of the full adder is transferred to a D flip-flop, the output of which is then used as the carry input for the next pair of significant bits.



Serial adder:

- The parallel adder uses registers with a parallel load, whereas the serial adder uses shift registers.
- The number of full-adder circuits in the parallel adder is equal to the number of bits in the binary numbers, whereas the serial adder requires only one full-adder circuit and a carry flip-flop.
- Excluding the registers, the parallel adder is a combinational circuit, whereas the serial adder is a sequential circuit.
-

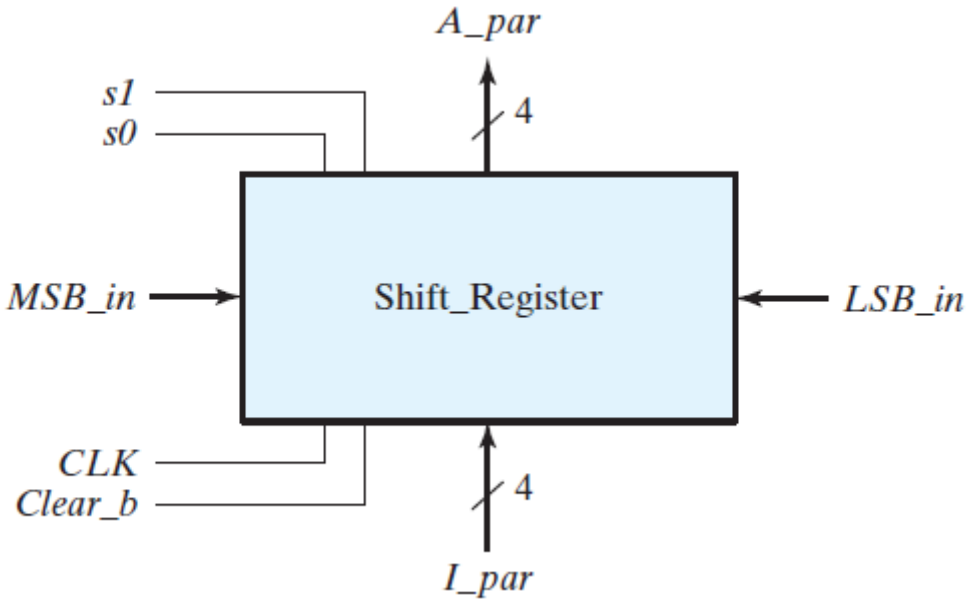
47.3 Universal Shift Register

- If the flip-flop outputs of a shift register are accessible, then information entered serially by shifting can be taken out in parallel from the outputs of the flip-flops.
- If a parallel load capability is added to a shift register, then data entered in parallel can be taken out in serial fashion by shifting the data stored in the register.
- The most general shift register has the following capabilities:

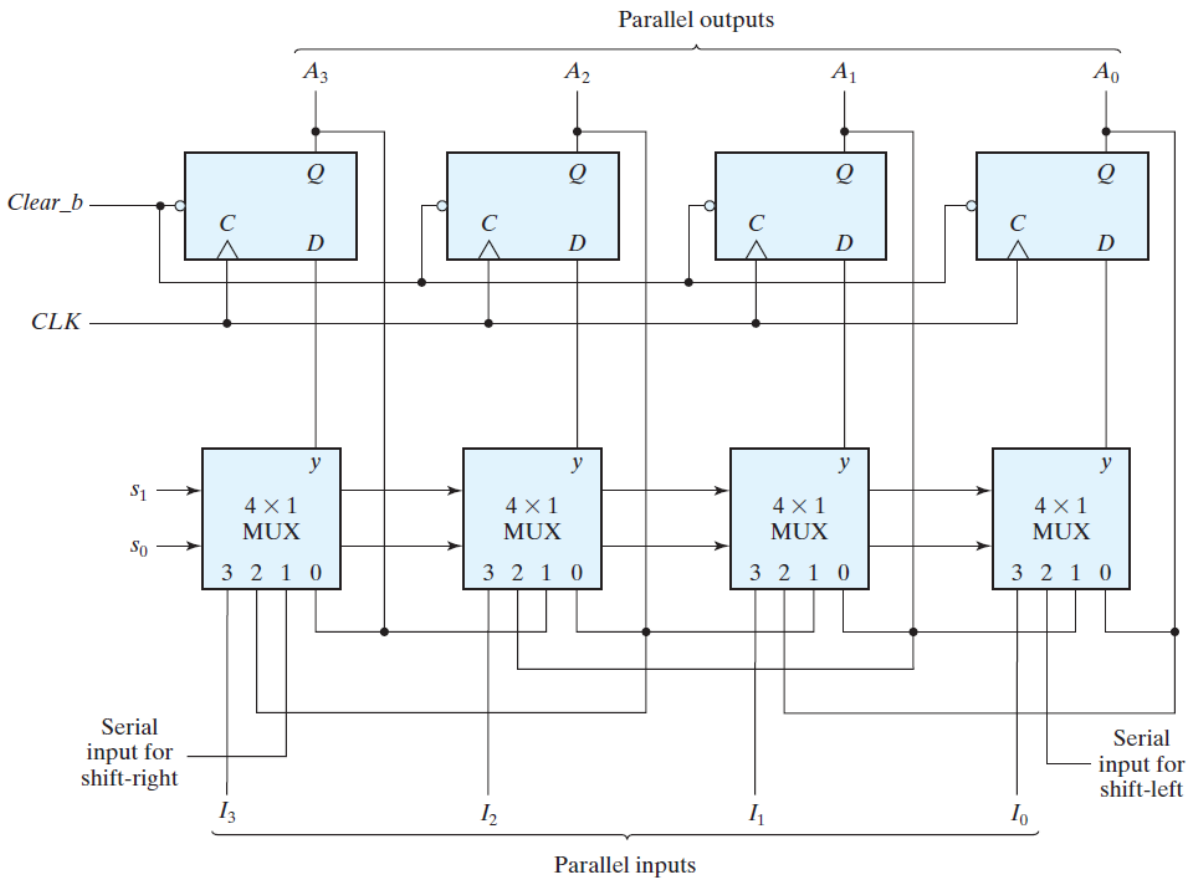
1. A clear control to clear the register to 0.
2. A clock input to synchronize the operations.
3. A shift-right control to enable the shift-right operation.
4. A shift-left control to enable the shift-left operation.
5. A parallel-load control to enable a parallel transfer and the n input lines associated with the parallel transfer.
6. n parallel output lines.
7. A control state that leaves the information in the register unchanged in response to the clock.

- A register capable of shifting in both the directions is a *bidirectional* shift register.
- If the register has both shifts and parallel-load capabilities, it is referred to as a *universal shift register*.

The block diagram symbol and the circuit diagram of a four-bit universal shift register is shown below:



Block diagram:



Circuit diagram:

- A_i: Parallel outputs
- I_i: Parallel inputs
- MSB_in: data entering for a shift-right operation
- LSB_in: data entering for a shift-left operation.
- Clear_b is an active-low signal that clears all of the flip-flops.
- S1, S0: The selection inputs which control the mode of operation of the register according to the following table:

s1	s0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

1. s1s0 = 00:
   - The present value of the register is applied to the D inputs of the flip-flops. Hence, the *output recirculates* to the input in this mode.
2. s1s0 = 00:
   - Terminal 1 of the multiplexer inputs has a path to the D inputs of the flip-flops. This causes a *shift-right operation.*
3. s1s0 = 00:

- Terminal 2 of the multiplexer inputs has a path to the D inputs of the flip-flops. This causes a *shift-left operation*.

4. s1s0 = 00:
   - the binary information on the parallel input lines is transferred into the register simultaneously during the next clock edge

   Shift registers are often used to interface digital systems situated remotely from each other.

---

48 Ripple Counters

A register that goes through a prescribed sequence of states upon the application of input pulses is called a *counter*.

- The sequence of states may follow the binary number sequence or any other sequence of states.
- A counter that follows the binary number sequence is called a binary counter .
- An n-bit binary counter consists of 'n' flip-flops and can count in binary from 0 through 2^n - 1.
- Counters are available in two categories:
  1. Ripple counters
  2. Synchronous counters

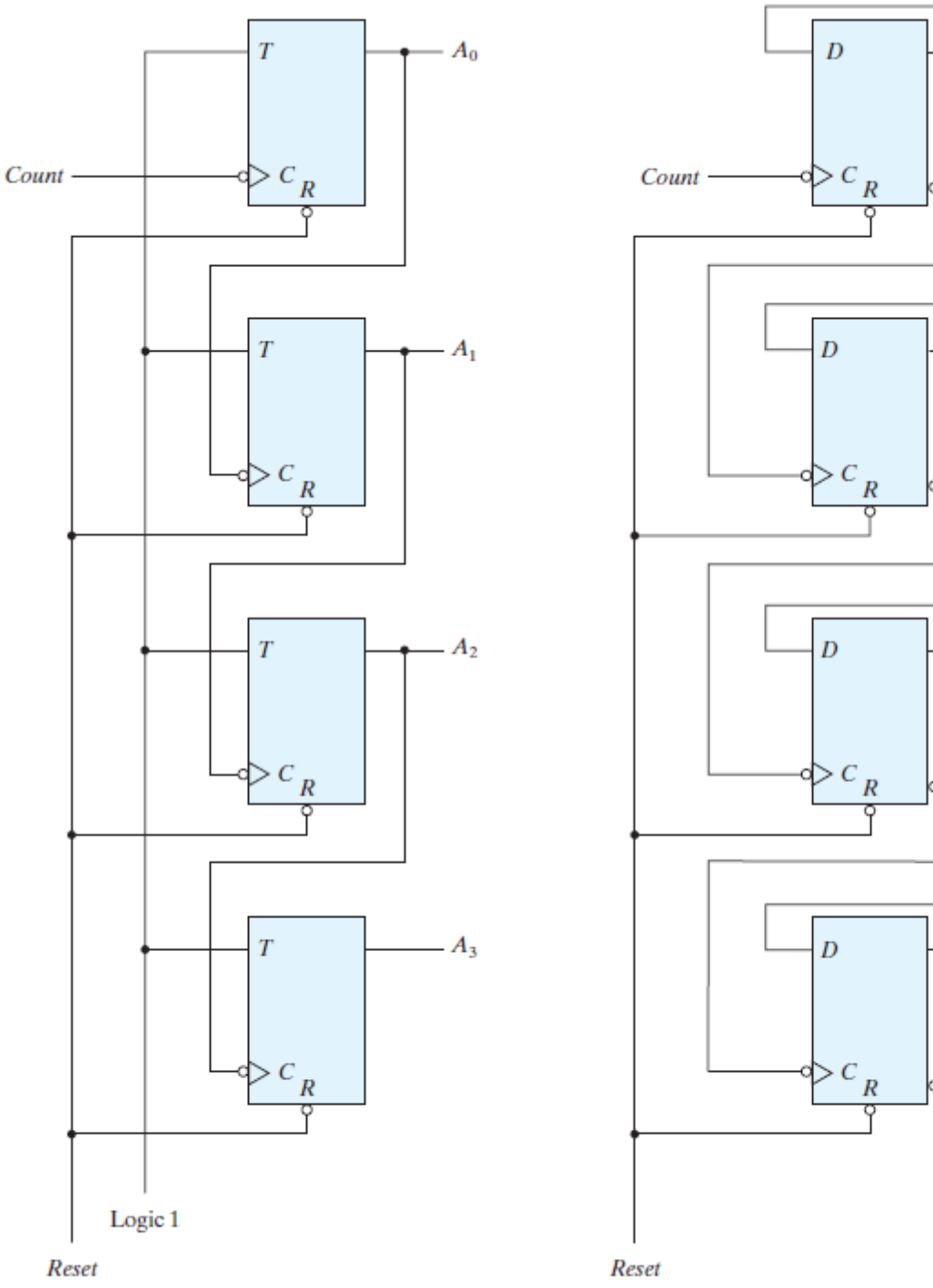
48.1 Ripple Counters

In a ripple counter, a flip-flop output transition serves as a source for triggering other flip-flops.

48.1.1 Binary Ripple Counter

A binary ripple counter consists of a series connection of complementing flip-flops, with the output of each flip-flop connected to the C input of the next higher order flip-flop. - The flip-flop holding the least significant bit receives the incoming count pulses.

- A complementing flip-flop can be obtained:
  1. T flip-flop
  2. D flip-flop with the complement output connected to the D input.
- The output of each flip-flop is connected to the C input of the next flip-flop in sequence.
- The output of each flip-flop is connected to the clk/control input of the next flip-flop.
- All the flip-flops are negative-edge triggered.



(a) With T flip-flops (b) With D flip-flops

Four-bit binary ripple counter:

- A binary counter with a reverse count is called a *binary countdown counter*. In a countdown counter, the binary count is decremented by 1 with every input count pulse.
- A *binary countdown counter* uses the same logic diagram of the binary ripple counter except all the flip-flops trigger on the positive edge of the clock.

48.2 Synchronous Counters

In *synchronous counters*, a common clock triggers all flip-flops simultaneously, unlike in a ripple counter.

48.2.1 Binary Counter

In a synchronous binary counter, the flip-flop in the least significant position is complemented with every pulse. - A flip-flop in any other position is complemented when all the bits in the lower significant positions are equal to 1. - Synchronous binary counters have a regular pattern and can be constructed with complementing flip-flops and gates. - The *C inputs* of all flip-flops are connected to a common clock. - The counter is enabled by *Count_enable*. - Binary counters are constructed most efficiently with T flip-flops because of their complement property. - The counter can be extended to any number of stages, with each stage having an additional flip-flop and an AND gate that gives an output of 1 if all previous flip-flop outputs are 1. - The flip-flops trigger on the positive edge of the clock. - The synchronous counter can be triggered with either the positive or the negative clock edge.

48.2.2 Up–Down Binary Counter

A synchronous countdown binary counter goes through the binary states in reverse order and repeats the count. - To achieve counting in the ascending manner, complemented output can be fed forward. - These two operations can be combined in one circuit to form a counter capable of counting either up or down. - The circuit of a Four-bit up–down binary counter using T flip-flops looks like this:

It has an up control input and a down control input. 1. Up = 1 & Down = 0: the circuit counts up. 2. Up = 0 & Down = 1: the circuit counts down. 3. Up = 0 & Down = 0: the circuit does not change state and remains in the same count. 4. Up = 1 & Down = 1: the circuit counts up.

> Note that the up input has priority over the down input.

48.3 Other Counters

- Counters can be designed to generate any desired sequence of states.
- Counters are used to generate timing signals to control the sequence of operations in a digital system.
- Few common examples of nonbinary counters are:
  1. Counter with Unused States: A binary counter using fewer states than the maximum possible number of states.
  2. Ring Counter: A *ring counter* is a circular shift register with only one flip-flop being set at any particular time; all others are cleared.
  3. Johnson Counter: A k-bit ring counter circulates a single bit among the flip-flops to provide k distinguishable states.

---

49 HDL for Registers and Counters

Revisit

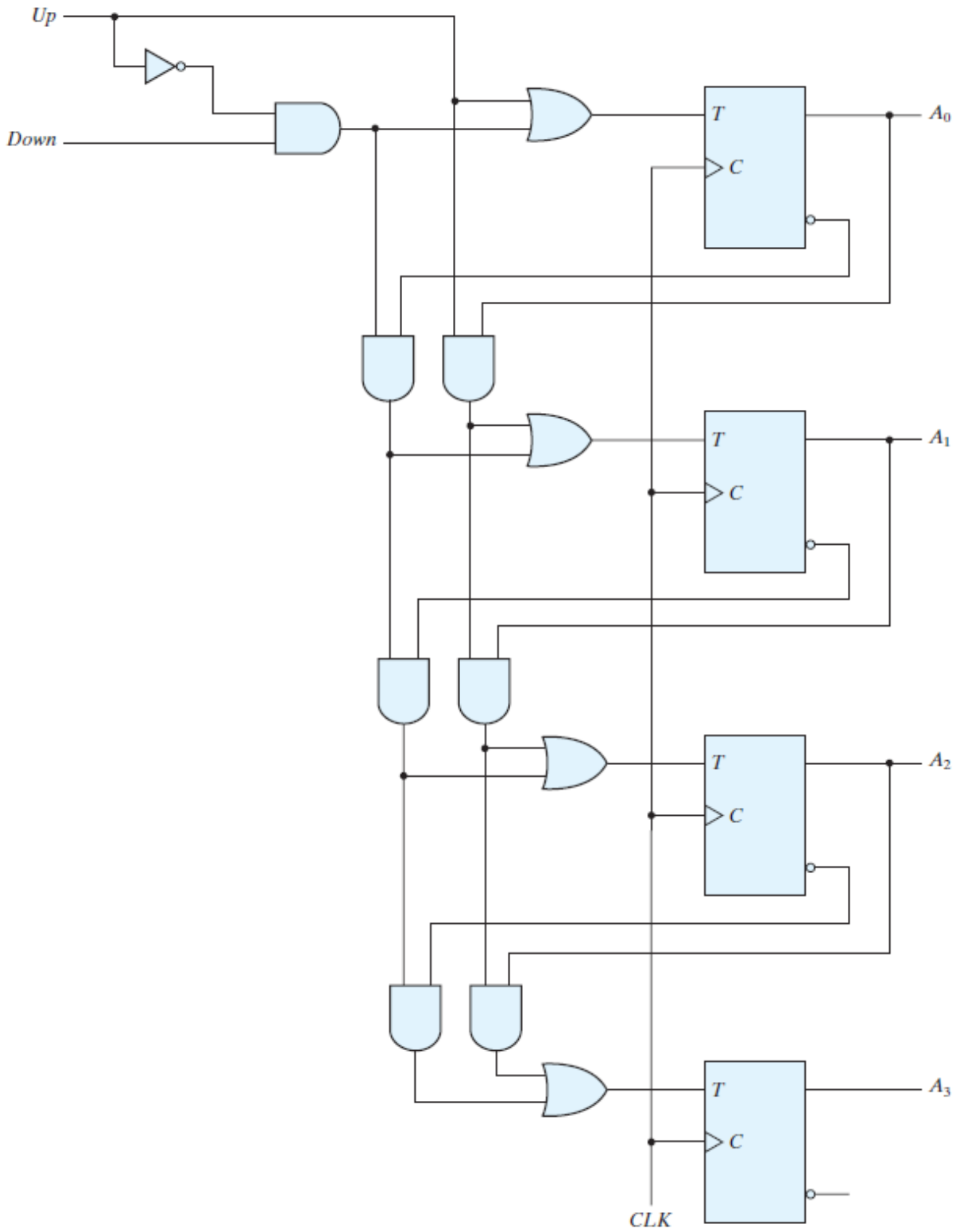


Figure 27: Four-bit up–down binary counter

50 Number conversion. Revise, watch youtube videos

- Binary Number representation
- Binary Number conversion
- Octal Hex Number representation
- Complement of numbers
- Signed Binary Number representation
- BCD code representation
- Gray code representation

51 Binary storage and registers

The binary information in a digital computer must have a physical existence in some medium for storing individual bits. A binary cell is a device that possesses two stable states and is capable of storing one bit (0 or 1) of information.

51.1 Registers

A register is a group of binary cells. A register with n cells can store any discrete quantity of information that contains n bits. The state of a register is an n-tuple of 1's and 0's, with each bit designating the state of one cell in the register. For example, a 16 bit register with the following binary content: 1100001111001001. A register with n cells can be in one of 2^n possible states.

51.2 Register Transfer

A digital system is characterized by its registers and the components that perform data processing. In digital systems, a register transfer operation is a basic operation that consists of a transfer of binary information from one set of registers into another set of registers.The transfer may be direct, from one register to another, or may pass through data-processing circuits to perform an operation.

52 Number conversion. Revise, watch youtube videos

- Binary Number representation
- Binary Number conversion
- Octal Hex Number representation
- Complement of numbers
- Signed Binary Number representation
- BCD code representation
- Gray code representation

53 Binary storage and registers

The binary information in a digital computer must have a physical existence in some medium for storing individual bits. A binary cell is a device that possesses two stable states and is capable of storing one bit (0 or 1) of information.

53.1 Registers

A register is a group of binary cells. A register with n cells can store any discrete quantity of information that contains n bits. The state of a register is an n-tuple of 1's and 0's, with each bit designating the state of one cell in the register. For example, a 16 bit register with the following binary content: 1100001111001001. A register with n cells can be in one of 2^n possible states.

53.2 Register Transfer

A digital system is characterized by its registers and the components that perform data processing. In digital systems, a register transfer operation is a basic operation that consists of a transfer of binary information from one set of registers into another set of registers.The transfer may be direct, from one register to another, or may pass through data-processing circuits to perform an operation.

Chapter 10 & Appendix - semiconductors and CMOS IC